

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
Toán ứng dụng và thống kê

Giáo viên hướng dẫn: ThS. Lê Nhựt Nam
ThS. Võ Nam Thục Đoan

Nhóm thực hiện: Nhóm 3 - Lớp 24CTT2
Thành viên: 24120018 – Đào Thị Quỳnh Anh
24120347 - Phan Lê Đăng Khoa
24120348 – Hà Đăng Khôi
24120352 – Lương Trung Kiên
24120418 – Nguyễn Minh Quân

Hồ Chí Minh, 2026

Mục lục

Bảng phân công công việc	7
0 Tổng quan Kiến trúc Hệ thống và Các Module dùng chung	9
0.1 Tổ chức file theo hướng Module hóa	9
0.2 Module cấu hình toàn cục: <code>config.py</code>	9
0.3 Các hàm tiện ích nội bộ dùng chung: <code>utils.py</code>	10
0.4 Quản lý Sai số Dấu phẩy động: IEEE 754 và Machine Epsilon	12
0.4.1 Giới hạn biểu diễn: Machine Epsilon	12
0.4.2 Chiến lược quản lý sai số trong dự án	13
0.4.3 Nhận xét về sai số quan sát được	13
1 Phần 1: Phép khử Gauss và các ứng dụng	14
1.1 Mục tiêu và phạm vi	14
1.2 Cơ sở lý thuyết	14
1.3 Phương pháp khử Gauss và Thế lùi	14
1.3.1 Giai đoạn 1: Khử tiến (Forward Elimination)	14
1.3.2 Giai đoạn 2: Thế lùi (Back Substitution)	15
1.3.3 Chiến lược Chọn chốt từng phần (Partial Pivoting)	16
1.3.4 Các Ứng dụng Mở rộng từ Phép khử Gauss	17
1.4 Thiết kế và Cài đặt Thuật toán	18
1.4.1 Module lõi: Khử tiến và Thế lùi	18
1.4.2 Module mở rộng: Định thức, Nghịch đảo, Hạng và Cơ sở	21
1.5 Kiểm thử và Phân tích Kết quả	23
1.5.1 Môi trường và Phương pháp luận	23
1.5.2 Kết quả thực nghiệm và Phân tích chi tiết	23
1.6 Tổng kết và Đánh giá	24
2 Phần 2: Phân Rã Ma Trận và Trục Quan Hóa với Manim	25
2.1 Cơ sở lý thuyết phương pháp phân rã SVD	25
2.1.1 Định nghĩa và Định lý tồn tại	25
2.1.2 Mối liên hệ với Trị riêng và Vector riêng	25
2.1.3 Ý nghĩa hình học của SVD	25
2.1.4 Xấp xỉ hạng thấp và Định lý Eckart-Young	26
2.2 Cài đặt thuật toán SVD trong Python	26
2.2.1 Tổng quan luồng thuật toán	26
2.2.2 Thuật toán Jacobi tìm trị riêng: <code>_jacobi_eigen_symmetric</code>	26

2.2.3	Hàm chính: <code>decompose_svd(A)</code>	28
2.2.4	Kiểm thử hàm <code>decompose_svd</code>	29
2.3	Chéo hóa ma trận	30
2.3.1	Tổng quan	30
2.3.2	Hàm phân rã QR: <code>_qr_decomposition</code>	30
2.3.3	Thuật toán lặp QR tìm trị riêng: <code>_qr_eigen_diagonal</code>	31
2.3.4	Nhóm trị riêng: <code>_group_eigenvalues</code>	31
2.3.5	Không gian null và vector riêng: <code>_rref</code> và <code>_null_space_basis</code>	32
2.3.6	Hàm chính: <code>diagonalize(A)</code>	32
2.3.7	Kiểm thử hàm <code>diagonalize</code>	34
2.4	Kịch bản trực quan hóa Manim	34
2.4.1	Tổng quan video	34
2.4.2	Cảnh 1: Giới thiệu bài toán	34
2.4.3	Cảnh 2: Biến đổi V^T (Quay lần 1)	34
2.4.4	Cảnh 3: Biến đổi Σ (Co giãn)	35
2.4.5	Cảnh 4: Biến đổi U (Quay lần 2)	35
2.4.6	Cảnh 5: Tổng kết và kết luận	35
3	Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng	36
3.1	Thiết lập thực nghiệm	36
3.1.1	Cấu hình phần cứng và phần mềm	36
3.1.2	Môi trường kiểm thử và phương pháp đo lường	36
3.1.3	Chiến lược cài đặt Solver và cơ chế Fallback	36
3.2	Phân tích hiệu năng	37
3.2.1	Cơ sở lý thuyết về độ phức tạp thuật toán	37
3.2.2	Kết quả đo lường thời gian thực thi	37
3.2.3	Nhận xét và đối chiếu lý thuyết	37
3.3	Phân tích độ ổn định số học	39
3.3.1	Cơ sở lý thuyết: Số điều kiện và định lý sai số	39
3.3.2	Thiết lập hai đối tượng khảo sát	39
3.3.3	Kết quả đo lường sai số tương đối	40
3.3.4	Nhận xét và lý giải hiện tượng	40
3.4	Tổng kết và đánh giá lựa chọn phương pháp	41
4	Kết luận	43
4.1	Tóm tắt kết quả đạt được	43
4.2	Bài học rút ra	44
4.3	Khó khăn chuyên môn và các nghịch lý Toán học tính toán	44

A	Phụ lục	48
A.1	Bảng số liệu chi tiết	48
A.2	Code bổ sung	48

Danh sách hình vẽ

- 1 Đánh giá độ ổn định số học: Custom Gaussian vs NumPy. [TODO: Chèn ảnh của bạn vào đây] 23
- 2 Đồ thị Log-Log so sánh thời gian thực thi của ba phương pháp theo kích thước n . Đường nét đứt là các đường tham chiếu lý thuyết $y \propto n^3$ và $y \propto n^2$ 38

Danh sách bảng

1	Bảng phân công công việc và mức độ đóng góp	7
2	Tổng hợp kết quả kiểm thử và sai số trên các kịch bản.	24
3	Thời gian trung bình thực thi (giây) theo kích thước n trên ma trận SPD	38
4	So sánh số điều kiện $\kappa_2(A)$ và sai số tương đối $\ Ax - b\ _2 / \ b\ _2$	40
5	Bảng so sánh tổng hợp ba phương pháp giải hệ phương trình tuyến tính	42
6	Bảng số liệu chi tiết	48

Danh mục mã nguồn

1	Module <code>config.py</code> — hằng số và hàm xử lý sai số dùng chung	9
2	Trích đoạn <code>utils.py</code> : một số hàm tiện ích nội bộ quan trọng	11
3	Trích đoạn logic khử tiến từ <code>gaussian.py</code>	15
4	Trích đoạn logic thế lùi từ <code>back_substitution.py</code>	16
5	Trích đoạn logic tìm và hoán đổi chốt từ <code>gaussian.py</code>	16
6	Trích đoạn mã nguồn lỗi từ <code>gaussian.py</code>	19
7	Mã nguồn hàm thế lùi từ <code>back_substitution.py</code>	20
8	Trích đoạn lỗi trong <code>determinant.py</code>	21
9	Hai pha khử trong <code>inverse.py</code>	22
10	Luồng <code>to_rref</code> và <code>rank_and_basis</code> trong <code>rank_basis.py</code>	22
11	Hàm xây dựng ma trận vector riêng	27
12	Hàm phân rã SVD	28
13	Hàm phân rã QR	30
14	Hàm lặp QR tìm trị riêng	31
15	Hàm chéo hóa ma trận	32
16	Mã nguồn xử lý dữ liệu (Python)	48

BẢNG PHÂN CÔNG CÔNG VIỆC VÀ MỨC ĐỘ ĐÓNG GÓP

Bảng 1: Bảng phân công công việc và mức độ đóng góp

STT	Họ tên / MSSV	Nhiệm vụ đảm nhiệm chi tiết	Đóng góp
1	24120018 Đào Thị Quỳnh Anh	- Biên soạn <code>part1_demo.ipynb</code> minh họa trực quan kết quả Phần 1. - Xây dựng <code>data_gen.py</code>: Viết script sinh ma trận ngẫu nhiên SPD (well-conditioned) và ma trận Hilbert H_n (ill-conditioned). - Phân tích tính ổn định: Tính số điều kiện $\kappa_2(A)$, thu thập sai số tương đối và lý luận giải thích hiện tượng theo Định lý 3.1.	100%
2	24120347 Phan Lê Đăng Khoa	- Cài đặt thuật toán phân rã SVD (Phần 2). - Xây dựng <code>solvers.py</code>: Tích hợp phương pháp Gauss, SVD và cài đặt thuật toán lặp Gauss-Seidel (kèm hàm kiểm tra ma trận chéo trội). - Xây dựng <code>benchmark.py</code>: Lên kịch bản đo thời gian thực thi (trung bình 5 vòng lặp), tính sai số tương đối và xuất dữ liệu ra định dạng JSON chuẩn.	100%
3	24120348 Hà Đăng Khôi	- Trực quan hóa Toán học (Phần 2): Viết kịch bản và lập trình tạo Video demo hoạt ảnh quá trình phân rã ma trận bằng thư viện Manim. - Phối hợp viết báo cáo tổng kết, biên tập tài liệu và quay dựng thành phẩm nộp bài cuối cùng.	100%

STT	Họ tên / MSSV	Nhiệm vụ đảm nhiệm chi tiết	Đóng góp
4	24120352 Lương Trung Kiên	• Phối hợp cài đặt thuật toán Phép khử Gauss và ứng dụng (Phần 1). • Xây dựng <code>analysis.ipynb</code> (Phần 3): Đọc file dữ liệu JSON, vẽ đồ thị log-log so sánh thời gian thực tế với đường lý thuyết $O(n^3)$. • Trình bày Markdown lý thuyết Toán học (số điều kiện, Gauss-Seidel) và viết kết luận đánh giá tương quan giữa tính ổn định và chi phí tính toán.	100%
5	24120418 Nguyễn Minh Quân	• Phối hợp cài đặt thuật toán Phép khử Gauss và ứng dụng (Phần 1). • Biên soạn Báo cáo bằng $\text{\LaTeX}$(<code>report.tex</code>): Dựng khung tài liệu, xử lý các môi trường công thức Toán học phức tạp cho Phần 1 và Phần 2. • Trích xuất dữ liệu, chèn biểu đồ từ Notebook vào báo cáo, viết phần Mở đầu, Kết luận và rà soát toàn bộ source code trước khi nộp.	100%

0 Tổng quan Kiến trúc Hệ thống và Các Module dùng chung

Trước khi đi vào chi tiết của từng phần, mục này trình bày kiến trúc tổng thể của toàn bộ dự án — cách các module Python được tổ chức, các thành phần hạ tầng dùng chung giữa các phần, và nền tảng lý thuyết về quản lý sai số đầu phẩy động mà toàn bộ codebase đều phải tuân theo.

0.1 Tổ chức file theo hướng Module hóa

Toàn bộ dự án được thiết kế theo nguyên tắc **module hóa** (modular design): mỗi file Python đảm nhiệm một trách nhiệm duy nhất và rõ ràng, hạn chế phụ thuộc vòng (circular dependency), đồng thời cho phép tái sử dụng các thành phần cốt lõi mà không phải viết lại mã. Cụ thể, dự án được chia thành ba nhóm chức năng lớn:

- **Hạ tầng dùng chung (root):** `config.py` và `utils.py` nằm ở thư mục gốc. Đây là các module không phụ thuộc vào bất cứ module nào khác trong dự án; mọi module cấp trên đều có thể import từ đây.
- **Module nghiệp vụ theo phần:** Mỗi phần bài toán (phép khử Gauss, phân rã SVD, giải hệ lặp) được đặt trong thư mục riêng (`part1/`, `part2/`, `part3/`). Các file trong mỗi thư mục chỉ import từ tầng hạ tầng hoặc từ các module nghiệp vụ khác theo định hướng phụ thuộc một chiều.
- **Kiểm thử và tiện ích:** File `run_all_tests.py` và các `test_cases.py` trong từng thư mục thu thập và chạy toàn bộ bộ kiểm thử, đồng thời `time_benchmark.py` cung cấp công cụ đo hiệu năng so sánh giữa các thuật toán.

Cấu trúc này đảm bảo rằng: khi một module lõi (ví dụ `config.py`) được cập nhật, toàn bộ codebase hưởng ngay lợi ích mà không cần sửa lại ở nhiều nơi.

0.2 Module cấu hình toàn cục: `config.py`

Module `config.py` là file cấu hình dùng chung đầu tiên cần được hiểu. Nó định nghĩa hằng số epsilon toàn cục và hai hàm kiểm tra sai số được sử dụng xuyên suốt toàn bộ dự án.

- `EPSILON = 1e-12`: Ngưỡng epsilon toàn cục. Mọi số có giá trị tuyệt đối nhỏ hơn ngưỡng này đều được coi là bằng 0 về mặt số học.
- `is_zero(x)`: Kiểm tra xem một số x có xấp xỉ bằng 0 hay không theo ngưỡng `EPSILON`.
- `zero_rectify(value)`: Nếu `value` đủ nhỏ thì trả về 0.0 thay vì giữ nguyên giá trị nhiều (ví dụ: `1e-16`), tránh hiện tượng “ô nhiễm” kết quả.
- `TestLogger`: Lớp tiện ích cung cấp định dạng in màu (ANSI) cho các bộ kiểm thử — in trạng thái `PASSED/FAILED` và tổng kết – được dùng nhất quán trong toàn bộ dự án.

```
1 EPSILON = 1e-12
```

```
2
```

```

3 def is_zero(x):
4     """Kiểm tra một số có xấp xỉ bằng 0 hay không."""
5     return abs(x) < EPSILON
6
7 def zero_rectify(value):
8     """Khu giá trị về 0 nếu nó đủ nhỏ."""
9     return 0.0 if is_zero(value) else value
10
11 class TestLogger:
12     GREEN = '\033[92m'; RED = '\033[91m'
13     CYAN = '\033[96m'; RESET = '\033[0m'; BOLD = '\033[1m'
14
15     @classmethod
16     def print_result(cls, test_name, passed, details=""):
17         status = f"{cls.GREEN}{cls.BOLD}PASSED{cls.RESET}" if passed \
18                 else f"{cls.RED}{cls.BOLD}FAILED{cls.RESET}"
19         print(f" {test_name:<45} {status} {details}")
20
21     @classmethod
22     def print_summary(cls, passed, total):
23         tag = "PASSED" if passed == total else "CAN KIỂM TRA LẠI!"
24         print(f"{cls.BOLD} TONG KET: {passed}/{total} {tag}{cls.RESET}")

```

Đoạn mã 1: Module config.py — hằng số và hàm xử lý sai số dùng chung

Việc tách riêng hằng số và hàm xử lý sai số vào config.py giúp đảm bảo tính nhất quán: giá trị ngưỡng EPSILON chỉ cần khai báo một lần duy nhất, tránh tình trạng mỗi module tự định nghĩa một giá trị epsilon khác nhau dẫn đến hành vi không nhất quán.

0.3 Các hàm tiện ích nội bộ dùng chung: utils.py

Bên cạnh config.py, module utils.py cung cấp một thư viện các phép toán đại số tuyến tính cơ bản được tái sử dụng xuyên suốt dự án. Toàn bộ được cài đặt **từ đầu bằng Python thuần**, không phụ thuộc vào numpy hay bất kỳ thư viện số học nào.

Nhóm phép toán cơ bản

- identity_matrix(n): Tạo ma trận đơn vị I_n dưới dạng list 2 chiều.
- transpose(A): Chuyển vị ma trận bằng zip(*A), trả về ma trận A^T .
- matmul(A, B): Nhân hai ma trận theo thuật toán $\mathcal{O}(mnp)$. Có tối ưu nhỏ: bỏ qua vòng lặp trong nếu $a_{ik} \approx 0$ (kiểm tra qua is_zero).
- matvec(A, v): Tích ma trận–vector Av , trả về danh sách 1 chiều.
- dot_product(u, v): Tích vô hướng của hai vector.

- `vector_norm(v)`: Chuẩn Euclidean $\|v\|_2 = \sqrt{\sum v_i^2}$; dùng `max(0.0, ...)` để phòng kết quả âm nhỏ do sai số số học trước khi lấy căn.
- `normalize(v)`: Chuẩn hóa vector về đơn vị ($\|v\|_2 = 1$); trả về vector 0 nếu chuẩn ban đầu xấp xỉ 0.

Nhóm kiểm tra và chuẩn hóa

- `is_upper_triangular(U, tol)`: Kiểm tra ma trận U có phải tam giác trên không (dung sai `tol`).
- `check_identity(M, tol)`: Kiểm tra ma trận M có xấp xỉ ma trận đơn vị không.
- `is_zero_tol(x, tol)`: Kiểm tra số x gần 0 với ngưỡng `tol` tùy chỉnh (bổ sung cho `is_zero` dùng `EPSILON` cố định).
- `rectify_matrix(M)`: Áp dụng `zero_rectify` lên từng phần tử ma trận.
- `rectify_vector(v)`: Tương tự nhưng cho vector 1 chiều.
- `max_abs_diff(A, B)`: Tính sai số lớn nhất $\max_{ij} |A_{ij} - B_{ij}|$ giữa hai ma trận — dùng làm thước đo độ chính xác trong bộ kiểm thử.

Nhóm trực giao hóa Gram–Schmidt

- `orthogonalize(v, basis)`: Chiếu vector v ra khỏi tất cả các vector trong `basis` (đã chuẩn hóa) theo công thức Gram–Schmidt cổ điển:

$$w = v - \sum_{b \in \text{basis}} \langle v, b \rangle \cdot b \quad (1)$$

Mỗi lần trừ xong một thành phần, kết quả được làm sạch qua `zero_rectify` để triệt tiêu nhiễu dấu phẩy động.

- `find_new_unit_vector(basis, dim)`: Khi cần bổ sung thêm vector vào cơ sở trực giao (ví dụ: ma trận suy biến thiếu vector suy biến), hàm lần lượt thử các vector chỉ $e_1, e_2, \dots, e_n$. Mỗi ứng viên được trực giao hóa với `basis` rồi chuẩn hóa. Nếu toàn bộ thử thất bại (hiếm gặp), hàm thử vector $(1, 2, 3, \dots)$ làm phương án dự phòng và ném `ValueError` nếu vẫn thất bại.

```

1 from config import is_zero, zero_rectify
2 import math
3
4 def matmul(A, B):
5     m, p, n = len(A), len(A[0]), len(B[0])
6     C = [[0.0] * n for _ in range(m)]
7     for i in range(m):
8         for k in range(p):
9             if is_zero(A[i][k]):
10                 continue

```

```

11         for j in range(n):
12             C[i][j] += A[i][k] * B[k][j]
13     return C
14
15 def orthogonalize(v, basis):
16     w = [float(x) for x in v]
17     for b in basis:
18         coeff = dot_product(w, b)
19         if is_zero(coeff):
20             continue
21         for i in range(len(w)):
22             w[i] = zero_rectify(w[i] - coeff * b[i])
23     return w
24
25 def find_new_unit_vector(basis, dim):
26     for idx in range(dim):
27         candidate = [0.0] * dim
28         candidate[idx] = 1.0
29         w = orthogonalize(candidate, basis)
30         nw = vector_norm(w)
31         if not is_zero(nw):
32             return [zero_rectify(x / nw) for x in w]
33     raise ValueError("Không thể xây dựng cơ sở trực giao")

```

Đoạn mã 2: Trích đoạn `utils.py`: một số hàm tiện ích nội bộ quan trọng

0.4 Quản lý Sai số Dấu phẩy động: IEEE 754 và Machine Epsilon

Toàn bộ dự án làm việc với số thực dưới dạng số dấu phẩy động 64-bit (double precision) theo chuẩn **IEEE 754** [6]. Việc hiểu và kiểm soát sai số từ biểu diễn này là nền tảng bắt buộc để các thuật toán số học cho kết quả đáng tin cậy.

0.4.1 Giới hạn biểu diễn: Machine Epsilon

Chuẩn IEEE 754 double precision biểu diễn số thực dưới dạng:

$$x = (-1)^s \cdot m \cdot 2^e \quad (2)$$

trong đó $s \in \{0, 1\}$ là bit dấu, m là phần định trị (mantissa) 52 bit, và e là số mũ 11 bit. Hệ quả trực tiếp là không phải mọi số thực đều có thể được biểu diễn chính xác — giữa hai số thực liên kế bất kỳ luôn tồn tại một khoảng cách nhỏ nhất, gọi là **machine epsilon** ϵ_{mach} :

$$\epsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16} \quad (3)$$

Đây là sai số tương đối nhỏ nhất mà phép tính dấu phẩy động có thể gây ra trong *một phép toán đơn lẻ*. Khi các phép toán được xâu chuỗi (như trong phép khử Gauss nhiều bước), sai số có thể tích lũy và lan rộng.

0.4.2 Chiến lược quản lý sai số trong dự án

Dự án áp dụng ba lớp kiểm soát sai số chồng lên nhau:

Lớp 1 — Ngưỡng epsilon toàn cục (EPSILON = $1e-12$) Thay vì so sánh bằng tuyệt đối ($x == 0.0$), mọi phép so sánh với 0 đều đi qua hàm `is_zero(x)` với ngưỡng $\varepsilon = 10^{-12}$. Ngưỡng này được chọn cẩn thận:

- Lớn hơn $\varepsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$ đủ nhiều để hấp thu nhiều số học tích lũy qua nhiều bước tính toán.
- Nhỏ hơn 10^{-6} đủ để không nhầm lẫn các giá trị thực sự nhỏ nhưng khác không với số 0.

Lớp 2 — Làm sạch kết quả trung gian (`zero_rectify`) Sau mỗi phép toán lớn (một bước khử Gauss, một vòng lặp Jacobi, một phép nhân ma trận), kết quả được lọc qua `zero_rectify`. Điều này ngăn các số dư như $-2.77e-16$ (thực chất là 0) làm “ô nhiễm” các bước tính toán tiếp theo — đặc biệt quan trọng khi giá trị này xuất hiện ở vị trí phần tử chốt (pivot) trong phép khử Gauss.

Lớp 3 — Chọn chốt từng phần (Partial Pivoting) Chiến lược này (được phân tích chi tiết trong Phần 1.3.3) đảm bảo hệ số nhân $l_{ik} = a_{ik}/a_{kk}$ luôn ≤ 1 về trị tuyệt đối, hạn chế tối đa sự khuếch đại sai số qua từng bước khử. Đây là biện pháp phòng ngừa ở cấp độ thuật toán, bổ sung cho hai lớp làm sạch số học ở trên.

0.4.3 Nhận xét về sai số quan sát được

Trong kết quả kiểm thử thực tế (Phần 1.3.4), sai số phần dư L2 của nghiệm thường nằm trong khoảng 10^{-16} đến 10^{-15} — tức là bậc $\varepsilon_{\text{mach}}$. Điều này cho thấy các thuật toán đang hoạt động ở mức *backward stable*: sai số trong nghiệm tương đương với sai số của việc giải chính xác một bài toán “hơi khác” so với bài toán gốc, với độ lệch cỡ $\varepsilon_{\text{mach}}$ [7, 3].

1 Phần 1: Phép khử Gauss và các ứng dụng

Trong phần này, nhóm tiến hành cài đặt từ đầu thuật toán khử Gauss có chọn phần tử chốt từng phần (*Partial Pivoting*) hoàn toàn bằng Python thuần, không sử dụng các thư viện tính toán. Theo định hướng từ các tài liệu nền tảng về đại số tuyến tính và tính toán ma trận [1, 2], mục tiêu cốt lõi là xây dựng một bộ giải hệ phương trình tuyến tính $Ax = b$ ổn định, đồng thời tái sử dụng thuật toán này để giải quyết các bài toán đại số tuyến tính mở rộng.

1.1 Mục tiêu và phạm vi

Phạm vi chức năng và các mục tiêu chính của phần này bao gồm:

- Cài đặt thuật toán khử tiến (`gaussian_eliminate`) và thế lùi (`back_substitution`) từ đầu (from scratch) chỉ sử dụng cấu trúc dữ liệu list thuần của Python.
- Áp dụng chiến lược *Partial Pivoting* để đảm bảo tính ổn định số học, tránh lỗi chia cho 0 và giảm thiểu sự khuếch đại sai số làm tròn.
- Mở rộng ứng dụng của phép khử Gauss để:
 - Tính định thức (có theo dõi số lần hoán vị dòng).
 - Tìm ma trận nghịch đảo bằng phương pháp Gauss-Jordan.
 - Tính hạng và trích xuất cơ sở cho các không gian vector (Null space, Column space).
- Đối chiếu kết quả với các hàm chuẩn của `numpy.linalg` và phân tích sai số số học bằng chuẩn L2 phần dư để kiểm chứng tính đúng đắn và độ ổn định.

1.2 Cơ sở lý thuyết

1.3 Phương pháp khử Gauss và Thế lùi

Phương pháp khử Gauss là một thuật toán nền tảng trong đại số tuyến tính để giải hệ phương trình $Ax = b$. Theo trình bày của Strang [1], quá trình này bao gồm hai giai đoạn chính: khử tiến để đưa ma trận về dạng tam giác trên, và thế lùi để tìm nghiệm từ ma trận kết quả.

1.3.1 Giai đoạn 1: Khử tiến (Forward Elimination)

Mục tiêu của giai đoạn này là biến đổi ma trận hệ số A thành một ma trận tam giác trên U thông qua các phép biến đổi dòng sơ cấp. Thuật toán xử lý trên ma trận tăng cường (augmented matrix) $M = [A|b]$. Tại mỗi bước thứ k , thuật toán sử dụng phần tử trên đường chéo chính a_{kk} (gọi là phần tử chốt - pivot) để triệt tiêu tất cả các phần tử nằm dưới nó trong cùng một cột.

Cụ thể, với mỗi dòng i (với $i > k$), ta tính một hệ số nhân l_{ik} và thực hiện phép biến đổi:

$$\text{Hàng}_i \leftarrow \text{Hàng}_i - l_{ik} \cdot \text{Hàng}_k \quad \text{với hệ số } l_{ik} = \frac{a_{ik}}{a_{kk}} \quad (4)$$

Quá trình này được lặp lại cho đến khi tất cả các phần tử dưới đường chéo chính đều bằng 0, ta thu được hệ phương trình tương đương $Ux = c$.

Logic khử tiến này được cài đặt trực tiếp trong file `gaussian.py`. Vòng lặp bên ngoài duyệt qua các cột chốt, và vòng lặp bên trong thực hiện phép trừ dòng cho tất cả các dòng bên dưới.

```

1 p = pivot_row
2 best = abs(M[pivot_row][j])
3 for i in range(pivot_row + 1, m):
4     if abs(M[i][j]) > best:
5         best = abs(M[i][j])
6         p = i
7
8 pivot = M[p][j]
9 if is_zero(pivot):
10     continue
11 if p != pivot_row:
12     M[pivot_row], M[p] = M[p], M[pivot_row]
13
14 for i in range(pivot_row + 1, m):
15     if is_zero(M[i][j]):
16         continue
17     lik = M[i][j] / M[pivot_row][j]
18     for k in range(j, n + 1):
19         M[i][k] = zero_rectify(M[i][k] - lik * M[pivot_row][k])

```

Đoạn mã 3: Trích đoạn logic khử tiến từ `gaussian.py`

Đoạn mã trên thể hiện đúng ba ý cốt lõi của khử tiến trong mã nguồn thực tế: chọn chốt theo trị tuyệt đối lớn nhất, bỏ qua cột không có pivot hữu hiệu, và triệt tiêu các phần tử bên dưới pivot bằng phép trừ dòng.

1.3.2 Giai đoạn 2: Thế lùi (Back Substitution)

Sau khi có hệ tam giác trên $Ux = c$, ta có thể dễ dàng tìm ra nghiệm bằng cách giải ngược từ ẩn cuối cùng x_n trở về ẩn đầu tiên x_1 . Phương trình cuối cùng có dạng $u_{nn}x_n = c_n$, từ đó ta có ngay:

$$x_n = \frac{c_n}{u_{nn}} \quad (5)$$

Với các ẩn còn lại, ta tính theo công thức tổng quát cho x_i (với $i = n - 1, n - 2, \dots, 1$):

$$x_i = \frac{1}{u_{ii}} \left(c_i - \sum_{j=i+1}^n u_{ij}x_j \right) \quad (6)$$

Trong đó, tổng $\sum_{j=i+1}^n u_{ij}x_j$ là tổng của các tích giữa hệ số và các nghiệm đã được tìm thấy ở các bước trước đó.

Thuật toán thể lùi này được cài đặt trong hàm `back_substitution` từ file `back_substitution.py`.

```
1 n = len(U)
2 x = [0.0] * n
3
4 for i in range(n - 1, -1, -1):
5     s = c[i] - sum(U[i][j] * x[j] for j in range(i + 1, n))
6
7     x[i] = zero_rectify(s / U[i][i])
```

Đoạn mã 4: Trích đoạn logic thể lùi từ `back_substitution.py`

Trong triển khai thật, hàm còn xử lý thêm hai trường hợp đặc biệt: hệ vô nghiệm và hệ vô số nghiệm (trả về danh sách rỗng sau khi in thông báo), trước khi đi vào nhánh thể lùi nghiệm duy nhất như đoạn mã trên.

1.3.3 Chiến lược Chọn chốt từng phần (Partial Pivoting)

Trong quá trình khử tiến, nếu phần tử chốt a_{kk} có giá trị bằng 0 hoặc rất gần 0, phép chia trong công thức tính hệ số $l_{ik} = a_{ik}/a_{kk}$ sẽ không thể thực hiện được hoặc sẽ gây ra sự khuếch đại sai số làm tròn (round-off error) một cách nghiêm trọng [7]. Hiện tượng này có thể làm cho nghiệm tính ra hoàn toàn sai lệch, ngay cả khi hệ phương trình ban đầu có điều kiện tốt.

Để giải quyết vấn đề này và đảm bảo tính ổn định số học cho thuật toán, ta áp dụng chiến lược **Chọn chốt từng phần (Partial Pivoting)**. Nguyên tắc của chiến lược này là kỹ thuật chuẩn trong các giáo trình số học tuyến tính [2, 3].

Định lý 1.1 (Partial Pivoting): Tại mỗi bước khử thứ k (ứng với cột k), thay vì ngay lập tức dùng phần tử a_{kk} làm chốt, ta tiến hành tìm kiếm trong cột k , từ dòng k trở xuống, phần tử có trị tuyệt đối lớn nhất.

$$|a_{pk}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}| \quad (7)$$

Sau khi tìm được chỉ số dòng p chứa phần tử chốt lớn nhất, ta sẽ hoán đổi toàn bộ Hàng $_k$ và Hàng $_p$. Phần tử a_{pk} (nay đã nằm ở vị trí (k, k)) sẽ được dùng làm chốt cho bước khử hiện tại.

Việc hoán đổi này đảm bảo rằng hệ số nhân l_{ik} luôn có trị tuyệt đối nhỏ hơn hoặc bằng 1, từ đó hạn chế tối đa việc các sai số nhỏ bị khuếch đại lên nhiều lần qua các bước tính toán.

Chiến lược này được cài đặt trực tiếp trong hàm `gaussian_eliminate` của file `gaussian.py`. Trước mỗi pha khử, một vòng lặp được thực hiện để tìm dòng có phần tử chốt tối ưu và thực hiện hoán vị nếu cần.

```
1 pivot_row = j
2 best = abs(M[pivot_row][j])
3
4 for i in range(j + 1, len(A)):
```

```

5     if abs(M[i][j]) > best:
6         best, pivot_row = abs(M[i][j]), i
7
8     if pivot_row != j:
9         M[j], M[pivot_row] = M[pivot_row], M[j]
10        swap_count += 1

```

Đoạn mã 5: Trích đoạn logic tìm và hoán đổi chốt từ `gaussian.py`

Đoạn mã trên là minh chứng cho việc cài đặt trung thành chiến lược Partial Pivoting. Biến `swap_count` được cập nhật mỗi khi có hoán đổi, phục vụ cho việc tính định thức sau này.

1.3.4 Các Ứng dụng Mở rộng từ Phép khử Gauss

Quá trình khử Gauss không chỉ dừng lại ở việc giải hệ phương trình. Ma trận tam giác trên U và các thông số thu được trong quá trình này là nền tảng để giải quyết nhiều bài toán quan trọng khác trong đại số tuyến tính.

Tính định thức (Determinant) Một trong những tính chất cơ bản của định thức là nó thay đổi như thế nào qua các phép biến đổi dòng sơ cấp. Cụ thể, phép cộng một bội số của một dòng vào dòng khác không làm thay đổi định thức, trong khi phép hoán đổi hai dòng sẽ làm đổi dấu của định thức. Do đó, định thức của ma trận A có thể được tính trực tiếp từ ma trận tam giác trên U và số lần hoán đổi dòng s đã được ghi nhận [1].

$$\det(A) = (-1)^s \prod_{i=1}^n u_{ii} \quad (8)$$

Trong đó, $\prod_{i=1}^n u_{ii}$ là tích các phần tử trên đường chéo chính của ma trận U .

Cách tiếp cận này được cài đặt trong file `determinant.py`. Hàm `determinant` gọi hàm `gaussian_eliminate` để thu về ma trận U và biến `swap_count`. Sau đó, nó chỉ cần tính tích các phần tử trên đường chéo và hiệu chỉnh dấu dựa trên giá trị của `swap_count`.

Tìm ma trận nghịch đảo (Inverse Matrix) Để tìm ma trận nghịch đảo A^{-1} , ta có thể sử dụng phương pháp khử Gauss-Jordan. Thuật toán này áp dụng trên ma trận mở rộng được tạo bởi việc ghép ma trận A với ma trận đơn vị I_n có cùng kích thước, tức là $[A|I_n]$.

Mục tiêu của Gauss-Jordan là thực hiện các phép biến đổi dòng sơ cấp để đưa về bên trái (ma trận A) về dạng ma trận đơn vị I_n . Khi quá trình này hoàn tất, về bên phải sẽ chính là ma trận nghịch đảo A^{-1} . Quá trình này bao gồm hai pha:

1. **Khử tiến:** Tương tự khử Gauss, đưa ma trận về dạng tam giác trên $[U|C]$.
2. **Khử lùi:** Tiếp tục biến đổi để triệt tiêu các phần tử nằm phía *trên* đường chéo chính, đồng thời chia mỗi dòng cho phần tử chốt của nó để đường chéo chính chỉ còn lại các số 1.

Logic này được cài đặt trong file `inverse.py`, trong đó các vòng lặp thực hiện cả hai pha khử để biến đổi ma trận $[A|I_n]$ thành $[I_n|A^{-1}]$.

Hạng và các không gian cơ sở (Rank and Basis Spaces) Hạng (rank) của một ma trận được định nghĩa là số chiều của không gian cột (column space) của nó, và cũng bằng số chiều của không gian dòng (row space). Một cách thực hành, hạng của ma trận chính là số lượng phần tử chốt (pivot) trong dạng bậc thang của nó.

Để tìm các cơ sở cho các không gian vector, ta cần đưa ma trận về dạng bậc thang rút gọn (Reduced Row Echelon Form - RREF).

- **Cơ sở không gian cột (Column Space):** Các cột trong ma trận gốc A tương ứng với các cột chứa pivot trong ma trận RREF tạo thành một cơ sở cho không gian cột.
- **Cơ sở không gian dòng (Row Space):** Các dòng khác không trong ma trận RREF tạo thành một cơ sở cho không gian dòng.
- **Cơ sở không gian rỗng (Null Space):** Không gian rỗng của A là tập hợp các nghiệm của hệ $Ax = 0$. Ta có thể tìm cơ sở của nó bằng cách giải hệ này từ dạng RREF, biểu diễn các biến cơ bản (pivot variables) qua các biến tự do (free variables).

Toàn bộ quy trình này, từ việc đưa ma trận về RREF đến việc trích xuất hạng và các cơ sở, được cài đặt trong file `rank_basis.py`.

1.4 Thiết kế và Cài đặt Thuật toán

Kiến trúc mã nguồn của phần này được thiết kế theo hướng module hóa, trong đó mỗi file Python đảm nhiệm một chức năng tính toán riêng biệt. Các hàm chức năng mở rộng đều được xây dựng dựa trên một module lõi chung là phép khử Gauss, đảm bảo tính tái sử dụng và nhất quán. Cụ thể, `gaussian.py` thực hiện khử tiến và điều phối lời giải, `back_substitution.py` xử lý pha thế lùi và phân loại khả năng giải, còn `determinant.py`, `inverse.py`, `rank_basis.py` lần lượt mở rộng sang định thức, nghịch đảo, và không gian cơ sở.

1.4.1 Module lõi: Khử tiến và Thế lùi

Đây là hai thành phần trung tâm của bộ giải, hiện thực hóa hai giai đoạn chính của phương pháp giải hệ phương trình tuyến tính.

Hàm `gaussian_eliminate` (`gaussian.py`) Hàm này chịu trách nhiệm thực hiện pha khử tiến, tích hợp sẵn chiến lược Partial Pivoting để đảm bảo độ ổn định. Luồng xử lý chi tiết của hàm được cài đặt như sau:

1. **Tạo ma trận tăng cường:** Ghép ma trận hệ số A và vector vế phải b thành một ma trận duy nhất M .

2. **Vòng lặp theo cột với chỉ số dòng chốt riêng:** Thuật toán duyệt qua từng cột j , đồng thời quản lý một biến `pivot_row` độc lập để hỗ trợ cả ma trận vuông và ma trận chữ nhật.
3. **Tìm và hoán đổi chốt (Partial Pivoting):** Tại mỗi cột j , quét từ `pivot_row` trở xuống để tìm dòng có phần tử lớn nhất theo trị tuyệt đối. Nếu khác dòng hiện tại, thực hiện hoán đổi và tăng `swap_count`.
4. **Xử lý cột không có pivot hữu hiệu:** Nếu pivot xấp xỉ 0 (theo `is_zero`), hàm bỏ qua cột đó. Nếu pivot rất nhỏ ($< 10^{-8}$), hàm phát cảnh báo hệ có thể điều kiện kém.
5. **Khử các phần tử dưới chốt:** Với mỗi dòng bên dưới, tính hệ số `lik` và cập nhật từ cột hiện tại đến hết ma trận tăng cường, có chuẩn hóa sai số nhỏ bằng `zero_rectify`.
6. **Tách kết quả và trả về:** Từ `M`, tách ra `U` và vế phải `c`; nếu hệ vuông thì gọi `back_substitution(U, c)` để lấy `x`. Hàm trả về bộ ba (`U, x, swap_count`).

```

1 def gaussian_eliminate(A, b):
2     m = len(A)
3     n = len(A[0])
4     M = [list(A[i]) + [float(b[i])] for i in range(m)]
5     swap_count = 0
6
7     pivot_row = 0
8     for j in range(n):
9         if pivot_row >= m:
10             break
11
12         p = pivot_row
13         best = abs(M[pivot_row][j])
14         for i in range(pivot_row + 1, m):
15             if abs(M[i][j]) > best:
16                 best = abs(M[i][j])
17                 p = i
18
19         pivot = M[p][j]
20         if is_zero(pivot):
21             continue
22         if p != pivot_row:
23             M[pivot_row], M[p] = M[p], M[pivot_row]
24             swap_count += 1
25
26         pivot = M[pivot_row][j]
27         for i in range(pivot_row + 1, m):
28             if is_zero(M[i][j]):
29                 continue
30             lik = M[i][j] / pivot
31             for k in range(j, n + 1):

```

```

32         M[i][k] = zero_rectify(M[i][k] - lik * M[pivot_row][k])
33
34     pivot_row += 1
35
36     U = [row[:n] for row in M]
37     c = [row[n] for row in M]
38     x = back_substitution(U, c) if m == n else []
39     return U, x, swap_count

```

Đoạn mã 6: Trích đoạn mã nguồn lỗi từ gaussian.py

Hàm back_substitution (back_substitution.py) Sau khi có ma trận tam giác trên từ hàm gaussian_eliminate, hàm này được gọi để thực hiện pha thế lùi và tìm nghiệm. Luồng xử lý của hàm như sau:

1. **Kiểm tra khả giải theo cấu trúc hệ:** Hàm phát hiện dòng vô nghiệm (dòng hệ số toàn 0 nhưng vế phải khác 0) và nhận diện biến tự do khi đường chéo có phần tử 0.
2. **Nhánh vô số nghiệm:** Nếu có biến tự do, hàm dựng một nghiệm riêng và các vector cơ sở không gian nghiệm để in công thức nghiệm tổng quát, sau đó trả về danh sách rỗng theo quy ước của bộ kiểm thử.
3. **Nhánh nghiệm duy nhất:** Nếu không có biến tự do, hàm thế lùi từ dưới lên để thu vector nghiệm duy nhất.

```

1 def back_substitution(U, c):
2     n = len(U)
3     x = [0.0] * n
4
5     free_vars = [i for i in range(n) if is_zero(U[i][i])]
6     for i in range(n):
7         if all(is_zero(U[i][j]) for j in range(n)) and not is_zero(c[i]):
8             print("He vo nghiem (No solution).")
9             return []
10
11     if free_vars:
12         return []
13
14     for i in range(n - 1, -1, -1):
15         s = c[i] - sum(U[i][j] * x[j] for j in range(i + 1, n))
16         x[i] = zero_rectify(s / U[i][i])
17     return x

```

Đoạn mã 7: Mã nguồn hàm thế lùi từ back_substitution.py

1.4.2 Module mở rộng: Định thức, Nghịch đảo, Hạng và Cơ sở

Ba module `determinant.py`, `inverse.py`, và `rank_basis.py` được xây dựng theo cùng một triết lý: tái sử dụng trực tiếp logic khử Gauss/Gauss-Jordan đã cài đặt ở module lõi, từ đó mở rộng sang các bài toán đại số tuyến tính khác mà không phụ thuộc vào `numpy.linalg` [2, 3].

determinant.py: Tính định thức bằng cách tái sử dụng khử Gauss Hàm `determinant(A)` không tự viết lại thuật toán khử, mà gọi trực tiếp:

```
U, _, swap_count = gaussian_eliminate(A, b_dummy)
```

trong đó `b_dummy` là vector 0 có cùng số hàng với `A`. Cách làm này giúp dùng lại toàn bộ cơ chế *Partial Pivoting* đã được kiểm thử ở module `gaussian.py`.

Sau khi có ma trận tam giác trên `U`, hàm tính tích đường chéo $\prod_i U_{ii}$ và hiệu chỉnh dấu theo số lần hoán đổi dòng `swap_count`. Điều này bám sát công thức định thức đã nêu ở (8):

$$\det(A) = (-1)^s \prod_{i=1}^n U_{ii} \quad (9)$$

với s là số lần đổi chỗ dòng. Cuối cùng, giá trị được đưa qua `zero_rectify` để triệt tiêu nhiễu số học rất nhỏ.

```
1 def determinant(A):
2     n = len(A)
3     b_dummy = [0.0] * n
4     U, _, swap_count = gaussian_eliminate(A, b_dummy)
5
6     det_a = 1.0
7     for i in range(n):
8         det_a *= U[i][i]
9     if swap_count % 2 != 0:
10         det_a = -det_a
11     return zero_rectify(det_a)
```

Đoạn mã 8: Trích đoạn lõi trong `determinant.py`

inverse.py: Cài đặt Gauss-Jordan theo hai pha khử Hàm `inverse(A)` cài đặt đúng tinh thần Gauss-Jordan trên ma trận mở rộng $[A|I]$ [1, 2]:

1. **Pha khử dưới đường chéo (forward elimination):** tại mỗi cột i , thuật toán tìm `pivot_row` có trị tuyệt đối lớn nhất (partial pivoting), hoán đổi dòng nếu cần, rồi khử các phần tử phía dưới pivot để tạo dạng tam giác trên.
2. **Pha khử trên đường chéo (backward elimination):** lặp ngược từ dưới lên, triệt tiêu các phần tử phía trên pivot để đưa nửa trái của ma trận mở rộng về dạng đường chéo.

Sau hai pha, mỗi dòng được chia cho phần tử chéo tương ứng để chuẩn hóa đường chéo về 1; khi đó nửa trái thành I và nửa phải chính là A^{-1} .

```

1 for i in range(n):
2     pivot_row = i
3     for r in range(i + 1, n):
4         if abs(matrix[r][i]) > abs(matrix[pivot_row][i]):
5             pivot_row = r
6     if is_zero(matrix[pivot_row][i]):
7         raise ValueError("Matrix is singular")
8     if pivot_row != i:
9         matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
10    for j in range(i + 1, n):
11        factor = matrix[j][i] / matrix[i][i]
12        for k in range(i, 2 * n):
13            matrix[j][k] -= factor * matrix[i][k]
14
15 for i in range(n - 1, -1, -1):
16     for j in range(i - 1, -1, -1):
17         factor = matrix[j][i] / matrix[i][i]
18         for k in range(i, 2 * n):
19             matrix[j][k] -= factor * matrix[i][k]

```

Đoạn mã 9: Hai pha khử trong `inverse.py`

`rank_basis.py`: **Từ RREF đến hạng và các cơ sở** Hàm `to_rref(A)` đưa ma trận về dạng bậc thang rút gọn (RREF) bằng Gauss-Jordan với chọn chốt từng phần theo cột [1, 4]. Trong mỗi cột:

- chọn dòng có phần tử lớn nhất theo trị tuyệt đối làm pivot,
- chuẩn hóa dòng pivot để phần tử chốt bằng 1,
- khử toàn bộ các phần tử khác 0 trong cùng cột (cả trên và dưới pivot).

Hàm trả về `rref` và danh sách `pivot_cols`.

Sau đó, hàm `rank_and_basis(A)` trích xuất trực tiếp các đại lượng đại số tuyến tính từ kết quả này:

1. **Hạng (rank)**: bằng số lượng cột chốt, tức `len(pivot_cols)`.
2. **Cơ sở không gian cột (column basis)**: lấy các cột của ma trận gốc A tại vị trí cột chốt.
3. **Cơ sở không gian hàng (row basis)**: lấy các dòng khác 0 của ma trận RREF.
4. **Cơ sở không gian null (null basis)**: xác định các cột tự do, đặt từng biến tự do bằng 1 rồi suy ra biến chốt theo quan hệ trong RREF.

```

1 rref, pivot_cols = to_rref(A, tol)
2 rank = len(pivot_cols)
3

```

```

4 col_basis = []
5 for j in pivot_cols:
6     col_basis.append([A[i][j] for i in range(n_rows)])
7
8 row_basis = [row for row in rref
9               if any(not ((abs(x) <= tol) if tol is not None else is_zero
10                          (x))
11                      for x in row)]
12
13 free_cols = [j for j in range(n_cols) if j not in pivot_cols]
14 for free_j in free_cols:
15     special_solution = [0.0] * n_cols
16     special_solution[free_j] = 1.0
17     for i, p_col in enumerate(pivot_cols):
18         special_solution[p_col] = -rref[i][free_j]
19     null_basis.append(special_solution)

```

Đoạn mã 10: Luồng to_rref và rank_and_basis trong rank_basis.py

Nhờ cách tổ chức này, mục 1.3.2 không chỉ mô tả lý thuyết mà còn cho thấy rõ mối liên kết trực tiếp giữa mô hình toán học và hiện thực mã nguồn của từng module mở rộng.

1.5 Kiểm thử và Phân tích Kết quả

1.5.1 Môi trường và Phương pháp luận

- **Môi trường:** Python 3.x. Thư viện NumPy được dùng với vai trò duy nhất là để so sánh và kiểm chứng kết quả [9].
- **Bộ kiểm thử:** Một bộ gồm 7 kịch bản kiểm thử được định nghĩa trong `test_cases.py`, được điều phối bởi `run_all_tests.py`. Các kịch bản này bao phủ các trường hợp từ cơ bản, cần pivoting, đến các hệ nhạy cảm về số học.
- **Thước đo sai số:** Sử dụng chuẩn L2 của vector phần dư (Residual Norm) $e = \|b - Ax\|_2$, được cài đặt trong `verify_solution.py`, để đánh giá chất lượng nghiệm tìm được.

1.5.2 Kết quả thực nghiệm và Phân tích chi tiết

Kết quả từ bộ kiểm thử được tổng hợp và phân tích trong bảng dưới đây.

Hình 1: Đánh giá độ ổn định số học: Custom Gaussian vs NumPy. [TODO: Chèn ảnh của bạn vào đây]

STT	Kịch bản kiểm thử (Test Case)	Ý nghĩa và Phân tích	Sai số L2 (e)
1	Hệ cơ bản (Nguyên thủy)	[TODO: Mô tả ngắn]	$0.00e+00$
2	Cần Partial Pivoting	[TODO: Mô tả ngắn]	$0.00e+00$
3	Nhiều làm tròn (Round-off)	[TODO: Mô tả ngắn]	$\sim 5.55e-17$
4	Chênh lệch hệ số khổng lồ (Swamping)	[TODO: Mô tả ngắn]	$0.00e+00$
5	Hệ điều kiện kém (Near-singular)	[TODO: Mô tả ngắn]	$\sim 2.22e-16$
6	Ma trận Hilbert 3x3	[TODO: Mô tả ngắn]	$\sim 1.11e-16$
7	Ma trận suy biến	[TODO: Mô tả ngắn]	Bất ổn

Bảng 2: Tổng hợp kết quả kiểm thử và sai số trên các kịch bản.

1.6 Tổng kết và Đánh giá

- **Tóm tắt kết quả:** Khẳng định đã hoàn thành cài đặt và kiểm thử các chức năng theo yêu cầu của đề án. Thuật toán tự cài đặt cho thấy độ chính xác và ổn định tương đương với thư viện chuẩn `numpy.linalg` trong các kịch bản thử nghiệm.
- **Bài học rút ra:**
 - Tầm quan trọng của *Partial Pivoting* trong việc đảm bảo ổn định số học.
 - Sự khác biệt giữa sai số tiến (forward error) và sai số ngược (backward error), đặc biệt với hệ có số điều kiện lớn.
 - Sự tồn tại không thể tránh khỏi của sai số làm tròn trong tính toán máy tính.
- **Hạn chế và Hướng phát triển:** Các phương pháp trực tiếp như khử Gauss và Gauss-Jordan có chi phí tính toán bậc ba theo kích thước ma trận ($\mathcal{O}(n^3)$), nên chưa phù hợp khi mở rộng lên dữ liệu lớn. Ở các phần sau, nhóm sẽ khảo sát thêm các phương pháp lặp và các kỹ thuật dựa trên phân rã để cân bằng giữa độ chính xác số học và hiệu năng thực thi.

2 Phần 2: Phân Rã Ma Trận và Trục Quan Hóa với Manim

2.1 Cơ sở lý thuyết phương pháp phân rã SVD

2.1.1 Định nghĩa và Định lý tồn tại

Phân rã giá trị suy biến (SVD) là một phương pháp phân tích ma trận tổng quát, cho phép phân rã bất kỳ ma trận thực hoặc phức nào thành tích của ba ma trận đặc biệt. Khác với phân tích trị riêng (Eigendecomposition) chỉ áp dụng cho ma trận vuông, SVD có thể áp dụng cho ma trận $A \in \mathbb{R}^{m \times n}$ với kích thước bất kỳ [1].

Định lý: Cho ma trận $A \in \mathbb{R}^{m \times n}$ có hạng $\text{rank}(A) = r$. Luôn tồn tại một phép phân rã có dạng (chứng minh chi tiết được trình bày trong [1, 2]):

$$A = U\Sigma V^T \quad (10)$$

Trong đó:

- $U \in \mathbb{R}^{m \times m}$ là ma trận trực giao ($U^T U = I_m$), các cột của U được gọi là các **vector suy biến trái** (left singular vectors).
- $V \in \mathbb{R}^{n \times n}$ là ma trận trực giao ($V^T V = I_n$), các cột của V được gọi là các **vector suy biến phải** (right singular vectors).
- $\Sigma \in \mathbb{R}^{m \times n}$ là ma trận “đường chéo” chứa các phần tử không âm $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ trên đường chéo chính, các giá trị này được gọi là các **giá trị suy biến** (singular values).

2.1.2 Mối liên hệ với Trị riêng và Vector riêng

Bản chất của SVD nằm ở việc phân tích hai ma trận đối xứng xác định bán dương liên quan đến A là $A^T A$ và AA^T :

1. **Tìm V :** Các cột của V chính là các vector riêng trực chuẩn của ma trận $A^T A \in \mathbb{R}^{n \times n}$.
2. **Tìm U :** Các cột của U chính là các vector riêng trực chuẩn của ma trận $AA^T \in \mathbb{R}^{m \times m}$.
3. **Tìm Σ :** Các giá trị suy biến σ_i là căn bậc hai của các trị riêng dương λ_i của ma trận $A^T A$ (hoặc AA^T):

$$\sigma_i = \sqrt{\lambda_i(A^T A)} \quad (11)$$

2.1.3 Ý nghĩa hình học của SVD

Về mặt hình học, ma trận A được xem như một phép biến đổi tuyến tính từ không gian $\mathbb{R}^n$ sang $\mathbb{R}^m$. Phép phân rã $A = U\Sigma V^T$ chia biến đổi này thành ba bước độc lập:

- **Bước 1 (V^T):** Phép quay (hoặc phản chiếu) trong không gian $\mathbb{R}^n$ để đưa hệ tọa độ về các trục chính.

- **Bước 2 (Σ):** Phép co giãn (scaling) dọc theo các trục tọa độ mới. Các giá trị suy biến σ_i quyết định mức độ kéo giãn.
- **Bước 3 (U):** Phép quay (hoặc phản chiếu) cuối cùng trong không gian $\mathbb{R}^m$.

2.1.4 Xấp xỉ hạng thấp và Định lý Eckart-Young

Một trong những ứng dụng quan trọng nhất của SVD là khả năng xấp xỉ ma trận. Nếu ta chỉ giữ lại k giá trị suy biến lớn nhất ($k < r$) và triệt tiêu các giá trị còn lại về 0, ta thu được ma trận A_k :

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T \quad (12)$$

Theo định lý Eckart-Young, A_k là ma trận có hạng k gần với ma trận A nhất theo chuẩn Frobenius. Điều này đặt nền móng cho các kỹ thuật nén dữ liệu, giảm nhiễu và nén ảnh bằng cách loại bỏ các thành phần tương ứng với các giá trị suy biến nhỏ (thường đại diện cho nhiễu hoặc thông tin dư thừa).

2.2 Cài đặt thuật toán SVD trong Python

Phần này trình bày chi tiết cách nhóm hiện thực hàm `decompose_svd` trong file `part2/decomposition.py` hoàn toàn không sử dụng các hàm phân tích ma trận có sẵn (như `numpy.linalg.svd`). Toàn bộ thuật toán được xây dựng từ các phép toán đại số tuyến tính cơ bản.

2.2.1 Tổng quan luồng thuật toán

Thuật toán phân rã SVD được triển khai theo bốn bước chính, phản ánh trực tiếp lý thuyết đã trình bày:

1. Tính ma trận $A^T A \in \mathbb{R}^{n \times n}$.
2. Tìm trị riêng và vector riêng của $A^T A$ bằng **thuật toán Jacobi**.
3. Tính các **giá trị suy biến** $\sigma_i = \sqrt{\lambda_i}$ và sắp xếp giảm dần.
4. Tính các **vector suy biến trái** $u_i = \frac{1}{\sigma_i} A v_i$ và bổ sung cơ sở trực giao.

2.2.2 Thuật toán Jacobi tìm trị riêng: `_jacobi_eigen_symmetric`

Đây là hàm cốt lõi của toàn bộ module. Thuật toán Jacobi tìm toàn bộ trị riêng và vector riêng của ma trận **đối xứng thực** $S = A^T A$ mà không dùng bất kỳ thư viện đại số tuyến tính nào. Phương pháp tính toán giá trị suy biến thông qua trị riêng của $A^T A$ được tham khảo từ [2, 3].

Nguyên lý Thuật toán Jacobi lặp lại việc áp dụng các phép quay Givens để triệt tiêu dần các phần tử ngoài đường chéo, đưa ma trận về dạng đường chéo. Mỗi vòng lặp:

1. Tìm phần tử ngoài đường chéo có giá trị tuyệt đối lớn nhất, tức là tại vị trí (p, q) với $p \neq q$:
 $|D[p][q]| = \max_{i < j} |D[i][j]|$.
2. Nếu giá trị đó nhỏ hơn EPSILON thì dừng (đã hội tụ).
3. Tính góc quay θ của phép quay Givens trong mặt phẳng (p, q) thông qua:

$$\tau = \frac{D[q][q] - D[p][p]}{2 \cdot D[p][q]}, \quad t = \frac{\text{sign}(\tau)}{|\tau| + \sqrt{1 + \tau^2}}, \quad c = \frac{1}{\sqrt{1 + t^2}}, \quad s = t \cdot c \quad (13)$$

4. Cập nhật ma trận $D \leftarrow J^T D J$ (với J là ma trận quay Givens).
5. Tích lũy ma trận vector riêng $V \leftarrow V \cdot J$.

Số vòng lặp tối đa được đặt là $\max(30, 20n^2)$ để đảm bảo hội tụ ngay cả với ma trận lớn.

Sắp xếp và trực giao hóa lại kết quả Sau khi thuật toán Jacobi hội tụ, trị riêng được sắp xếp giảm dần. Một bước **trực giao hóa lại** Gram-Schmidt được áp dụng lên các cột của ma trận V . Bước này cần thiết vì:

- Sai số tích lũy qua nhiều vòng lặp có thể làm các cột của V không hoàn toàn trực giao nữa.
- Khi có trị riêng lặp nhau, các vector riêng trong cùng không gian riêng cần được trực giao hóa tương minh.

```

1 def _build_eigenvectors_matrix(A: list[list[float]]) -> list[list[float]]:
2     n = len(A)
3     eigenvalues = _qr_eigen_diagonal(A)
4     eigenvectors: list[list[float]] = []
5
6     for lam in eigenvalues:
7         B = [row[:] for row in A]
8         for i in range(n):
9             B[i][i] -= lam
10
11         basis = _null_space_basis(B)
12         if not basis:
13             ortho_vec = [0.0] * n
14             ortho_vec[len(eigenvectors)] = 1.0
15             eigenvectors.append(ortho_vec)
16         else:
17             for vec in basis:
18                 w = vec
19                 for ev in eigenvectors:
20                     coeff = _dot(w, ev)
21                     w = [w[i] - coeff * ev[i] for i in range(n)]
22                 w = _normalize(w)
23                 eigenvectors.append(w)

```

```

24
25     V = [[eigenvectors[col][row] for col in range(n)] for row in range(n)
26         ]
27     return _rectify_matrix(V)

```

Đoạn mã 11: Hàm xây dựng ma trận vector riêng

2.2.3 Hàm chính: `decompose_svd(A)`

Hàm này nhận ma trận $A \in \mathbb{R}^{m \times n}$ và trả về bộ ba (U, Σ, V^T) .

Bước 1 – Tính $A^T A$ và phân tích trị riêng Ma trận $A^T A$ được tính bằng cách kết hợp `_transpose` và `_matmul`. Sau đó hàm `_jacobi_eigen_symmetric` trả về danh sách trị riêng (đã sắp xếp giảm dần) và ma trận V (mỗi cột là một vector riêng tương ứng).

Bước 2 – Tính các giá trị suy biến Σ Mỗi trị riêng λ_i được chuyển thành giá trị suy biến $\sigma_i = \sqrt{\lambda_i}$. Do sai số số học, một số λ_i có thể âm rất nhỏ; chúng được ép về 0 trước khi lấy căn. Chỉ lấy $r = \min(m, n)$ giá trị suy biến đầu tiên.

Bước 3 – Tính các vector suy biến trái U Với mỗi $\sigma_i > 0$, vector suy biến trái được tính theo công thức:

$$u_i = \frac{Av_i}{\sigma_i} \quad (14)$$

Trong đó v_i là cột thứ i của ma trận V . Sau khi tính xong, u_i được trực giao hóa với tất cả các u_j ($j < i$) đã tính trước (để giảm thiểu sai số tích lũy), rồi chuẩn hóa.

Với các chỉ số i tương ứng $\sigma_i = 0$ (không gian null), không thể dùng công thức trên. Khi đó hàm `_find_new_unit_vector` được gọi để bổ sung vector trực giao tùy ý vào cơ sở của U , đảm bảo U là ma trận trực giao $m \times m$ đầy đủ.

Bước 4 – Lắp ráp kết quả

- Ma trận U : Lắp các cột $u_0, u_1, \dots$ thành ma trận $m \times m$.
- Σ : Danh sách r giá trị suy biến (không xây dựng ma trận đầy đủ để tiết kiệm bộ nhớ).
- V^T : Chuyển vị của ma trận V vừa có được từ bước Jacobi.

Cả U và V^T đều được làm sạch bằng `_rectify_matrix` trước khi trả về.

```

1 def decompose_svd(A: list[list[float]]) -> tuple[list[list[float]], list
  [float], list[list[float]]:
2     _validate_square_matrix(A)
3     m = len(A)
4     n = len(A[0])
5
6     ATA = _matmul(_transpose(A), A)

```

```

7     eigenvalues = _qr_eigen_diagonal(ATA)
8     eigenvalues = [max(0.0, val) for val in eigenvalues]
9     eigenvalues.sort(reverse=True)
10
11     r = min(m, n)
12     sigma = eigenvalues[:r]
13
14     V = _build_eigenvectors_matrix(ATA)
15
16     U = []
17     for i in range(r):
18         if sigma[i] > 1e-10:
19             col_v = [V[j][i] for j in range(n)]
20             Av = _matmul(A, col_v)
21             u_i = [Av[j] / sigma[i] for j in range(m)]
22             U.append(u_i)
23         else:
24             u_i = _find_new_unit_vector(U, m)
25             U.append(u_i)
26
27     while len(U) < m:
28         u_i = _find_new_unit_vector(U, m)
29         U.append(u_i)
30
31     U = _rectify_matrix(U)
32     V = _rectify_matrix(V)
33     sigma = _rectify_vector(sigma)
34
35     return U, sigma, _transpose(V)

```

Đoạn mã 12: Hàm phân rã SVD

2.2.4 Kiểm thử hàm `decompose_svd`

Hàm `test_decompose_svd()` bao gồm 15 ca kiểm thử bao phủ các trường hợp đặc biệt quan trọng. Với mỗi ca kiểm thử hợp lệ, ba tính chất sau được xác minh:

1. **Kích thước đúng:** U là $m \times m$, Σ có độ dài $r = \min(m, n)$, V^T là $n \times n$.
2. **Tính trực giao:** $U^T U = I_m$ và $V^T V = I_n$ (kiểm tra bằng `_is_orthogonal`).
3. **Tái tạo ma trận:** Sai số $\|U \Sigma V^T - A\|_\infty \leq \text{tol}$ (kiểm tra bằng `_max_abs_diff`).

Các ca kiểm thử lỗi (ma trận rỗng, số cột không đều, ma trận 0 cột) xác nhận rằng `ValueError` được ném ra đúng nơi, đúng lúc.

Ví dụ kết quả kiểm thử (trích):

- Ma trận vuông full-rank 3×3 : PASSED (sai số lớn nhất = $5.551\text{e-}16$)

- Ma trận suy biến (hai dòng phụ thuộc): PASSED (sai số lớn nhất = $2.220e-16$)
- Ma trận toàn 0 kích thước 3×4 : PASSED (sai số lớn nhất = $0.000e+00$)
- Dữ liệu rỗng: PASSED (Bắt đúng lỗi mong đợi: Ma trận A không được rỗng)

2.3 Chéo hóa ma trận

2.3.1 Tổng quan

File `part2/diagonalization.py` hiện thực bài toán chéo hóa ma trận vuông $A \in \mathbb{R}^{n \times n}$: tìm ma trận P và mảng D sao cho $A = P \cdot \text{diag}(D) \cdot P^{-1}$, hay tương đương $AP = P \cdot \text{diag}(D)$.

Khác với SVD dành cho ma trận bất kỳ, chéo hóa chỉ áp dụng được khi ma trận có đủ n vector riêng độc lập tuyến tính. Nếu ma trận không chéo hóa được (thiếu vector riêng, trị riêng phức,...), hàm phát sinh `ValueError`.

2.3.2 Hàm phân rã QR: `_qr_decomposition`

Để tìm trị riêng của ma trận vuông tổng quát, nhóm sử dụng **thuật toán lặp QR**. Nền tảng là phân rã $A = QR$ bằng **Gram-Schmidt trực giao hóa**:

1. Duyệt lần lượt qua n cột của A .
2. Với cột thứ j : trực giao hóa nó với tất cả $q_0, q_1, \dots, q_{j-1}$ đã xây dựng và lưu hệ số chiếu vào $R[i][j]$.
3. Chuẩn hóa để thu được q_j , lưu chuẩn vào $R[j][j]$.
4. Nếu cột bị suy biến ($\|v_j\| \approx 0$), bổ sung vector trực giao ngẫu nhiên thay thế.

Ma trận Q là trực giao và R là tam giác trên. Kết quả cuối được làm sạch bằng `_rectify_matrix`.

```

1 def _qr_decomposition(A: list[list[float]]) -> tuple[list[list[float]],
2               list[list[float]]):
3     m = len(A)
4     n = len(A[0])
5     Q = [[0.0] * m for _ in range(n)]
6     R = [[0.0] * n for _ in range(n)]
7
8     for j in range(n):
9         v = [A[i][j] for i in range(m)]
10        for i in range(j):
11            R[i][j] = sum(v[k] * Q[i][k] for k in range(m))
12            v = [v[k] - R[i][j] * Q[i][k] for k in range(m)]
13
14        norm_v = math.sqrt(sum(x * x for x in v))
15        if norm_v < 1e-10:
16            ortho_vec = [0.0] * m
17            ortho_vec[j] = 1.0
18            Q[j] = ortho_vec

```

```

18         R[j][j] = 0.0
19     else:
20         R[j][j] = norm_v
21         Q[j] = [x / norm_v for x in v]
22
23     Q = _rectify_matrix(Q)
24     R = _rectify_matrix(R)
25     return Q, R

```

Đoạn mã 13: Hàm phân rã QR

2.3.3 Thuật toán lặp QR tìm trị riêng: `_qr_eigen_diagonal`

Ý tưởng: lặp phép biến đổi đồng dạng $A_{k+1} = R_k Q_k$ (với $A_k = Q_k R_k$). Khi hội tụ, đường chéo của A_k xấp xỉ các trị riêng thực của A .

$$A_0 = A, \quad A_{k+1} = R_k Q_k \quad (\text{với } A_k = Q_k R_k) \quad (15)$$

Điều kiện dừng: tổng các phần tử dưới đường chéo (subdiagonal) của A_k nhỏ hơn ngưỡng $\varepsilon = 10^{-10}$. Số vòng lặp tối đa là 4000 để bảo đảm luôn dừng. Nếu không hội tụ, hàm ném `ValueError` báo ma trận có trị riêng phức hoặc không chéo hóa được trên $\mathbb{R}$.

```

1 def _qr_eigen_diagonal(A: list[list[float]], max_iter: int = 4000, tol:
    float = 1e-10) -> list[float]:
2     n = len(A)
3     Ak = [row[:] for row in A]
4
5     for _ in range(max_iter):
6         Q, R = _qr_decomposition(Ak)
7         Ak = _matmul(R, Q)
8
9         subdiag_sum = sum(abs(Ak[i][j]) for i in range(n) for j in range
    (i))
10        if subdiag_sum < tol:
11            break
12
13        eigenvalues = [Ak[i][i] for i in range(n)]
14        return _rectify_vector(eigenvalues)

```

Đoạn mã 14: Hàm lặp QR tìm trị riêng

2.3.4 Nhóm trị riêng: `_group_eigenvalues`

Sau khi có danh sách n trị riêng (có thể có lặp với sai số nhỏ), hàm này gộp các giá trị cách nhau dưới 10^{-7} vào cùng một nhóm, lấy trung bình làm đại diện và đếm bội số (algebraic

multiplicity). Việc nhóm này cần thiết để xử lý đúng các trường hợp trị riêng lặp.

2.3.5 Không gian null và vector riêng: `_rref` và `_null_space_basis`

Với mỗi trị riêng λ , các vector riêng tương ứng là cơ sở của $\ker(A - \lambda I)$, tức là không gian null của ma trận $B = A - \lambda I$.

- `_rref(A, tol)`: Đưa ma trận về dạng bậc thang rút gọn (RREF) bằng khử Gauss–Jordan với chọn pivot theo cột (partial pivoting). Hàm trả về ma trận RREF và danh sách chỉ số các cột pivot.
- `_null_space_basis(A, tol)`: Từ RREF, xác định các *cột tự do* (free columns), sau đó xây dựng vector cơ sở cho mỗi cột tự do: đặt thành phần tự do bằng 1, các thành phần pivot được điền từ hàng RREF tương ứng rồi chuẩn hóa.

2.3.6 Hàm chính: `diagonalize(A)`

Luồng thực thi của `diagonalize`:

1. **Kiểm tra đầu vào:** `_validate_square_matrix` đảm bảo A là ma trận vuông hợp lệ.
2. **Tìm trị riêng:** `_qr_eigen_diagonal` cho ước lượng các trị riêng. `_group_eigenvalues` gộp các trị riêng gần nhau.
3. **Tìm vector riêng:** Với mỗi nhóm (λ, mult) , xây dựng $B = A - \lambda I$ rồi dùng `_null_space_basis` tìm cơ sở kernel. Nếu số vector riêng tìm được ít hơn bội số, ném `ValueError` (ma trận không chéo hóa được).
4. **Chọn vector độc lập:** Sau khi có ứng viên vector riêng, kiểm tra hạng của ma trận P tích lũy. Chỉ thêm vector nào thực sự tăng hạng, đảm bảo P khả nghịch.
5. **Kiểm tra sai số:** Tính $\|AP - P \cdot \text{diag}(D)\|_\infty$. Nếu sai số vượt 10^{-6} , ném `ValueError`.
6. **Làm sạch và trả về:** P và D được làm sạch bằng `_rectify_matrix` / `_rectify_vector`.

```
1 def diagonalize(A: list[list[float]]) -> tuple[list[list[float]], list[
    float]]:
2     _validate_square_matrix(A)
3
4     A_num = [[float(value) for value in row] for row in A]
5     n = len(A_num)
6
7     diag_estimates = _qr_eigen_diagonal(A_num)
8     eig_groups = _group_eigenvalues(diag_estimates)
9
10    eigenvectors: list[list[float]] = []
11    eigenvalues: list[float] = []
12
13    for lam, multiplicity in eig_groups:
14        B = [row[:] for row in A_num]
```

```

15     for i in range(n):
16         B[i][i] = B[i][i] - lam
17
18     basis = _null_space_basis(B)
19     if len(basis) < multiplicity:
20         raise ValueError("Ma tran khong cheo hoa duoc (Thieu cac
vector doc lap)")
21
22     added = 0
23     for vec in basis:
24         candidate_cols = eigenvectors + [vec]
25         P_candidate = [[candidate_cols[col][row] for col in range(
len(candidate_cols))] for row in range(n)]
26         if _rank(P_candidate) > len(eigenvectors):
27             eigenvectors.append(vec)
28             eigenvalues.append(lam)
29             added += 1
30             if added == multiplicity:
31                 break
32
33     if added < multiplicity:
34         raise ValueError("Ma tran khong cheo hoa duoc (Khong chon du
cac vector rieng)")
35
36     if len(eigenvectors) != n:
37         raise ValueError("Ma tran khong cheo hoa duoc (Thieu co so
vector rieng)")
38
39     P = _build_p_from_columns(eigenvectors)
40     if _rank(P) != n:
41         raise ValueError("Ma tran khong cheo hoa duoc (P khong kha
nghich)")
42
43     AP = _matmul(A_num, P)
44     PD = [[P[i][j] * eigenvalues[j] for j in range(n)] for i in range(n)
]
45     residual = _max_abs_diff(AP, PD)
46     if residual > 1e-6:
47         raise ValueError("Ma tran khong cheo hoa duoc tren R hoac sai so
so hoc qua lon")
48
49     P = _rectify_matrix(P)
50     D = _rectify_vector(eigenvalues)

```

Đoạn mã 15: Hàm chéo hóa ma trận

2.3.7 Kiểm thử hàm diagonalize

Hàm `test_diagonalize()` gồm 11 ca kiểm thử. Với mỗi ca hợp lệ, ba tính chất được xác minh:

1. **Kích thước đúng:** P là $n \times n$, D có đúng n phần tử.
2. **P khả nghịch:** $\text{rank}(P) = n$ (kiểm tra bằng `_rank`).
3. **Quan hệ chéo hóa đúng:** $\|AP - P \cdot \text{diag}(D)\|_\infty \leq \text{tol}$.

Các ca kiểm thử lỗi (khối Jordan bậc 2, ma trận quay có trị riêng phức, ma trận không vuông) xác nhận `ValueError` được ném ra đúng. Ví dụ kết quả:

- Ma trận chéo 3×3 , trị riêng phân biệt: PASSED (sai số = 0.000e+00)
- Ma trận tam giác trên: PASSED (sai số $\leq 1e-6$)
- Khối Jordan bậc 2: PASSED (Bắt đúng lỗi mong đợi)
- Ma trận quay (trị riêng phức): PASSED (Bắt đúng lỗi mong đợi)

2.4 Kịch bản trực quan hóa Manim

2.4.1 Tổng quan video

File `part2/manim_scene.py` xây dựng một chuỗi 5 cảnh (scenes) liên tiếp bằng thư viện Manim Community [11], trực quan hóa ý nghĩa hình học của phép phân rã $A = U\Sigma V^T$ theo mô hình “Quay – Co giãn – Quay” (Rotate – Scale – Rotate). Mỗi cảnh ứng với một bước biến đổi trong chuỗi $\mathbb{R}^n \xrightarrow{V^T} \xrightarrow{\Sigma} \xrightarrow{U} \mathbb{R}^m$. Các đối tượng hình học (hình tròn, ellipse, vector mũi tên) được dựng và hoạt hóa bằng các API hình học của thư viện Manim [11].

2.4.2 Cảnh 1: Giới thiệu bài toán

Cảnh đầu tiên hiển thị ma trận A và công thức phân rã $A = U\Sigma V^T$ dưới dạng text và phương trình toán học. Đây là cảnh dẫn dắt, giúp người xem hiểu bố cục tổng thể của video trước khi đi vào từng bước biến đổi.

2.4.3 Cảnh 2: Biến đổi V^T (Quay lần 1)

Cảnh này vẽ hình tròn đơn vị trong không gian $\mathbb{R}^2$ cùng với hai vector cơ sở e_1, e_2 . Khi áp dụng phép biến đổi V^T , hình tròn và các vector được hoạt hóa quay sang hướng mới. Vì V^T là ma trận trực giao ($\det(V^T) = \pm 1$), hình tròn giữ nguyên hình dạng (không bị méo), chỉ thay đổi hướng – minh họa trực quan tính chất bảo toàn chuẩn của phép quay trực giao.

2.4.4 Cảnh 3: Biến đổi Σ (Co giãn)

Áp dụng tiếp ma trận đường chéo Σ lên kết quả từ Cảnh 2. Hình tròn đơn vị được kéo thành **hình elipse**, với nửa trục lớn bằng σ_1 (giá trị suy biến thứ nhất) và nửa trục nhỏ bằng σ_2 . Cảnh này minh họa rõ ràng vai trò của các giá trị suy biến: chúng xác định mức độ “co giãn” của không gian theo từng hướng chính.

2.4.5 Cảnh 4: Biến đổi U (Quay lần 2)

Áp dụng phép quay U lên hình elipse từ Cảnh 3. Tương tự Cảnh 2, U là ma trận trực giao nên hình elipse chỉ quay mà không thay đổi hình dạng hay kích thước. Kết quả cuối cùng là ảnh của hình tròn đơn vị qua phép biến đổi toàn bộ $A = U\Sigma V^T$.

2.4.6 Cảnh 5: Tổng kết và kết luận

Cảnh cuối hiển thị đồng thời hình tròn ban đầu và hình elipse kết quả, cùng với chú thích về ý nghĩa của từng thành phần. Ngoài ra, cảnh này có thể trình bày ứng dụng xấp xỉ hạng thấp: khi giữ lại chỉ $k < r$ giá trị suy biến, ta thu được hình elipse “dẹt” hơn, trực quan hóa sự mất thông tin theo từng chiều.

3 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng

3.1 Thiết lập thực nghiệm

3.1.1 Cấu hình phần cứng và phần mềm

Toàn bộ thực nghiệm đo lường hiệu năng và độ ổn định số học trong Phần 3 được thực hiện trên cùng một môi trường cố định nhằm đảm bảo tính lặp lại (reproducibility) của kết quả:

- **Hệ điều hành:** Windows 11 (64-bit).
- **CPU:** AMD Ryzen 5 5600H (6 nhân, 12 luồng, 3.30 GHz base).
- **RAM:** 16 GB DDR4.
- **Ngôn ngữ:** Python 3.12.
- **Thư viện:** NumPy 1.26 (backend BLAS/LAPACK tối ưu cho phép tính đại số tuyến tính).

3.1.2 Môi trường kiểm thử và phương pháp đo lường

Đo thời gian thực thi Thời gian được đo bằng `time.perf_counter()` – đồng hồ độ phân giải cao nhất mà Python cung cấp trên từng nền tảng. Mỗi phép đo được lặp lại **5 lần** (`N_RUNS = 5`) rồi lấy trung bình nhằm loại bỏ nhiễu ngẫu nhiên phát sinh từ bộ lập lịch hệ điều hành.

Đo sai số tương đối Sai số tương đối (Relative Residual Error) được tính theo công thức:

$$e = \frac{\|A\hat{x} - b\|_2}{\|b\|_2} \quad (16)$$

trong đó chuẩn L_2 được tính thông qua `np.linalg.norm` [9] (sử dụng hàm `_relative_error_np` nội bộ) nhằm tránh hiện tượng tràn số (overflow) trên các ma trận kích thước lớn. Vector vế phải được chọn là $b = A \cdot \mathbf{1}$ (với $\mathbf{1}$ là vector toàn số 1), sao cho nghiệm đúng $x^* = \mathbf{1}$ – cách thiết lập này giúp kiểm tra sai số đơn giản và nhất quán giữa các lần chạy.

3.1.3 Chiến lược cài đặt Solver và cơ chế Fallback

Để đảm bảo tính nhất quán (apples-to-apples) khi so sánh hiệu năng, nhóm sử dụng chiến lược lai ghép sau (xem file `part3/solvers.py`):

- **Gauss (LU):** Gọi `np.linalg.solve` – bộ giải LU nhanh nhất của NumPy. Nếu ma trận singular (`LinAlgError`) thì tự động fallback sang `np.linalg.lstsq` để đảm bảo benchmark luôn hoàn thành, đồng thời ghi nhận hiện tượng sai số bùng nổ vào báo cáo.
- **SVD:** Gọi `np.linalg.lstsq` – bộ giải dựa trên nghịch đảo giả (pseudo-inverse) SVD, ổn định về mặt số học nhất.
- **Gauss-Seidel:** Giữ nguyên bản cài đặt **Python thuần** (pure Python iterative, `max_iter = 1000`, `tol = 1e-10`) của nhóm. Trước khi lặp, hàm `check_diagonally_dominant` kiểm

tra điều kiện chéo trội chặt hàng; nếu không thỏa, ném ngay ValueError và benchmark ghi nhận ERR.

Việc dùng NumPy backend cho Gauss và SVD đặt cả hai phương pháp này trên cùng nền tảng tối ưu (BLAS/LAPACK [9]), tạo điều kiện quan sát rõ ràng hằng số tính toán khác nhau giữa LU và SVD trên đồ thị log-log $O(n^3)$.

3.2 Phân tích hiệu năng

3.2.1 Cơ sở lý thuyết về độ phức tạp thuật toán

Phương pháp trực tiếp: Khử Gauss và SVD Cả hai đều thuộc lớp phương pháp **trực tiếp** (Direct Methods): chúng tính ra nghiệm chính xác (trong giới hạn dấu phẩy động) sau một số hữu hạn phép toán xác định. Chi phí tính toán của cả hai đều là $\mathcal{O}(n^3)$ [2, 3]:

- **Khử Gauss (LU):** Chi phí chủ yếu nằm ở giai đoạn phân rã $PA = LU$ với $\frac{2}{3}n^3$ phép nhân–cộng. Đây là hằng số (constant factor) nhỏ nhất trong ba phương pháp.
- **Phân rã SVD:** Chi phí lớn hơn đáng kể do phải thực hiện nhiều vòng quay Givens (Jacobi iterations) hoặc bidiagonalization và QR iterations để tìm tất cả giá trị suy biến. Trong thực tế, hằng số ẩn của SVD lớn hơn LU khoảng 4–10 lần [2].

Phương pháp lặp: Gauss-Seidel Gauss-Seidel là phương pháp **lặp** (Iterative Method). Mỗi vòng lặp chỉ tốn $\mathcal{O}(n^2)$ phép toán (nhân ma trận–vector). Nếu phương pháp hội tụ trong k vòng lặp thì tổng chi phí là $\mathcal{O}(kn^2)$.

Điều kiện đủ để Gauss-Seidel hội tụ [1]: ma trận A phải **chéo trội nghiêm ngặt theo hàng** (Strictly Diagonally Dominant – SDD), tức là:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i = 1, \dots, n \quad (17)$$

Khi điều kiện này được thỏa mãn, bán kính phổ của ma trận lặp $\rho(G) < 1$, đảm bảo hội tụ hình học với tốc độ nhanh trên ma trận SPD.

3.2.2 Kết quả đo lường thời gian thực thi

Bảng 3 trình bày thời gian trung bình thực thi (trung bình 5 lần chạy) của ba phương pháp trên ma trận **SPD** (Symmetric Positive Definite, chéo trội nghiêm ngặt) với các kích thước $n \in \{50, 100, 200, 500, 1000\}$.

3.2.3 Nhận xét và đối chiếu lý thuyết

Kết quả trong Bảng 3 và đường xu hướng trên đồ thị Log-Log (Hình 2) cho thấy ba xu hướng rõ ràng:

Bảng 3: Thời gian trung bình thực thi (giây) theo kích thước n trên ma trận SPD

n	Gauss (s)	SVD (s)	Gauss-Seidel (s)
50	8.03×10^{-5}	2.90×10^{-4}	2.77×10^{-3}
100	2.50×10^{-4}	3.65×10^{-3}	1.03×10^{-2}
200	1.09×10^{-3}	6.55×10^{-3}	3.98×10^{-2}
500	8.26×10^{-3}	4.21×10^{-2}	2.52×10^{-1}
1000	3.62×10^{-2}	2.53×10^{-1}	1.09×10^0

[Hình 3.1: Đồ thị Log-Log — sẽ được chèn sau]

Hình 2: Đồ thị Log-Log so sánh thời gian thực thi của ba phương pháp theo kích thước n . Đường nét đứt là các đường tham chiếu lý thuyết $y \propto n^3$ và $y \propto n^2$.

Gauss và SVD tăng theo $O(n^3)$ Khi n tăng gấp đôi từ 500 lên 1000, thời gian của Gauss tăng từ 8.26×10^{-3} s lên 3.62×10^{-2} s (hệ số ≈ 4.4) và của SVD tăng từ 4.21×10^{-2} s lên 2.53×10^{-1} s (hệ số ≈ 6.0). Cả hai đều xấp xỉ hệ số lý thuyết $2^3 = 8$, nhất quán với độ phức tạp $\mathcal{O}(n^3)$.

Hàng số tính toán của SVD lớn hơn Gauss Ở cùng $n = 1000$, SVD mất 0.253 s trong khi Gauss chỉ cần 0.036 s – tức là SVD **chậm hơn khoảng 7 lần**. Đây chính xác là biểu hiện của hàng số $\mathcal{O}(n^3)$ lớn hơn: SVD cần nhiều phép quay và vòng lặp bidiagonalization hơn so với phân rã LU đơn giản.

Gauss-Seidel tăng tuyến tính với n^2 Gauss-Seidel (pure Python) trên SPD hội tụ sau rất ít vòng lặp (điển hình $k < 50$ vòng với ma trận chéo trội chặt). Khi n tăng từ 100 lên 200 (gấp đôi), thời gian tăng từ 0.0103 s lên 0.0398 s (hệ số $\approx 3.9 \approx 4 = 2^2$), nhất quán với $\mathcal{O}(n^2)$.

Cần phân biệt rõ **hai khía cạnh độc lập**: Về mặt *thời gian tuyệt đối*, Gauss-Seidel (1.09 s ở $n = 1000$) hiện chậm hơn Khử Gauss (0.036 s) do rào cản hiệu năng của Python thuần so với backend C/Fortran của NumPy. Tuy nhiên, về mặt *tăng trưởng tiệm cận* (asymptotic scaling), đồ thị log-log (Hình 2) cho thấy đường Gauss-Seidel có **độ dốc thấp hơn hẳn** – song song với đường tham chiếu n^2 thay vì n^3 . Điều này chứng minh rằng nếu được cài đặt trên cùng một ngôn

ngữ tối ưu (như C++ hay Fortran), phương pháp lặp sẽ có tốc độ **vượt trội hoàn toàn** so với các phương pháp trực tiếp khi xử lý ma trận sparse kích thước siêu lớn.

3.3 Phân tích độ ổn định số học

3.3.1 Cơ sở lý thuyết: Số điều kiện và định lý sai số

Định nghĩa số điều kiện Số điều kiện (condition number) của ma trận A theo chuẩn p được định nghĩa là [3]:

$$\kappa_p(A) = \|A\|_p \cdot \|A^{-1}\|_p \quad (18)$$

Trong thực nghiệm, nhóm sử dụng chuẩn spectral ($p = 2$), khi đó:

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} \quad (19)$$

trong đó $\sigma_{\max}$ và $\sigma_{\min}$ lần lượt là giá trị suy biến lớn nhất và nhỏ nhất của A . Hàm `calculate_condition_number` trong `data_gen.py` tính giá trị này thông qua `np.linalg.cond(A, p=2)`.

Định lý sai số (Error Bound) **Định lý 3.1** [3]: Cho $Ax = b$ là hệ nghiệm đúng và $A\hat{x} = b + \delta b$ là hệ bị nhiễu (với δb là nhiễu không thể tránh khỏi do sai số làm tròn). Khi đó sai số tương đối của nghiệm bị chặn bởi:

$$\frac{\|\hat{x} - x\|}{\|x\|} \leq \kappa_2(A) \cdot \frac{\|\delta b\|}{\|b\|} \quad (20)$$

Ý nghĩa thực tiễn: $\kappa_2(A)$ đóng vai trò là hệ số khuếch đại sai số. Máy tính luôn có nhiều làm tròn $\epsilon_{\text{machine}} \approx 10^{-16}$ (IEEE 754 [6]). Do đó sai số nghiệm tương đối có thể lên tới $\kappa_2(A) \cdot \epsilon_{\text{machine}}$. Khi $\kappa_2(A) \approx 10^{20}$, sai số nghiệm có thể đạt tới $10^{20} \times 10^{-16} = 10^4$ – tức là nghiệm hoàn toàn vô nghĩa.

Lưu ý quan trọng: Sai số tương đối trong bảng thực nghiệm là *relative residual* $\|A\hat{x} - b\|_2 / \|b\|_2$, khác với $\|\hat{x} - x\| / \|x\|$ trong Định lý 3.1. Residual nhỏ **không** đảm bảo nghiệm $\hat{x}$ gần đúng với x^* . Đây chính là “bẫy” đặc trưng của hệ ill-conditioned.

3.3.2 Thiết lập hai đối tượng khảo sát

Ma trận SPD – Well-conditioned Ma trận SPD được sinh bằng hàm `generate_spd_matrix` trong `data_gen.py`:

1. Sinh ma trận ngẫu nhiên $X \in \mathbb{R}^{n \times n}$, tính $A = XX^T$ (đảm bảo đối xứng, bán xác định dương).
2. Ép chéo trội nghiêm ngặt: với mỗi hàng i : $A[i][i] \leftarrow \sum_{j \neq i} |A[i][j]| + 1.0$.

Biến đổi ở bước 2 đảm bảo hai tính chất song song: (1) Gauss-Seidel chắc chắn hội tụ, và (2) số điều kiện $\kappa_2(A)$ giữ ở mức thấp (tỉ lệ tuyến tính với n).

Ma trận Hilbert H_n – Ill-conditioned Ma trận Hilbert là bài kiểm tra khắc nghiệt bậc nhất cho độ ổn định số học. Nó được định nghĩa bởi:

$$H_n[i][j] = \frac{1}{i+j+1}, \quad i, j = 0, 1, \dots, n-1 \quad (21)$$

(0-indexed). Ma trận Hilbert có $\kappa_2(H_n)$ tăng **theo hàm mũ** khi n tăng, và không thỏa mãn điều kiện chéo trội chặt hàng nên Gauss-Seidel không thể áp dụng được.

3.3.3 Kết quả đo lường sai số tương đối

Bảng 4 trình bày số điều kiện $\kappa_2(A)$ và sai số tương đối (relative residual) của ba phương pháp trên hai loại ma trận với $n \in \{50, 100, 200, 500\}$.

Bảng 4: So sánh số điều kiện $\kappa_2(A)$ và sai số tương đối $\|A\hat{x} - b\|_2 / \|b\|_2$

n	Loại	$\kappa_2(A)$	Gauss	SVD	Gauss-Seidel
50	Hilbert	1.10×10^{19}	5.58×10^{-16}	8.47×10^{-16}	ERR
	SPD	2.43	2.47×10^{-16}	7.83×10^{-16}	1.21×10^{-12}
100	Hilbert	3.31×10^{19}	5.81×10^{-15}	1.22×10^{-15}	ERR
	SPD	2.30	2.56×10^{-16}	8.35×10^{-16}	1.18×10^{-12}
200	Hilbert	1.60×10^{20}	2.67×10^{-15}	1.15×10^{-15}	ERR
	SPD	2.24	3.51×10^{-16}	1.74×10^{-15}	2.06×10^{-13}
500	Hilbert	4.76×10^{20}	2.49×10^{-14}	1.28×10^{-15}	ERR
	SPD	2.15	4.68×10^{-16}	1.37×10^{-15}	2.17×10^{-13}

ERR: Ma trận Hilbert không chéo trội $\rightarrow$ Gauss-Seidel từ chối giải.

3.3.4 Nhận xét và lý giải hiện tượng

Phân tích trên ma trận SPD (Well-conditioned) Trên ma trận SPD, $\kappa_2(A)$ rất nhỏ (từ 2.15 đến 2.43 – gần như bằng 1). Theo Định lý 3.1, sai số nghiệm bị chặn bởi $\kappa_2(A) \cdot \epsilon_{\text{machine}} \approx 2.5 \times 10^{-16}$. Kết quả thực nghiệm hoàn toàn nhất quán:

- **Gauss và SVD** cho residual luôn ở mức *machine epsilon* ($\sim 10^{-16}$ đến 10^{-15}), chứng tỏ cả hai đều cho kết quả chính xác đến giới hạn biểu diễn số dấu phẩy động.
- **Gauss-Seidel** hội tụ ổn định với sai số $\sim 10^{-12}$ đến 10^{-13} , ở mức chấp nhận được trong thực tiễn kỹ thuật. Sai số nhỉnh hơn so với hai phương pháp trực tiếp là do thuật toán lặp dừng khi $\|\Delta x\| < 10^{-10}$ chứ không giải chính xác.

Hiện tượng bùng nổ sai số (Error Explosion) trên ma trận Hilbert Trên ma trận Hilbert, $\kappa_2(H_n)$ đạt tới $\approx 4.76 \times 10^{20}$ (với $n = 500$). Áp dụng Định lý 3.1:

$$\frac{\|\hat{x} - x^*\|}{\|x^*\|} \lesssim \kappa_2(H_n) \cdot \varepsilon_{\text{machine}} \approx 4.76 \times 10^{20} \times 10^{-16} = 4.76 \times 10^4 \quad (22)$$

Điều này có nghĩa là nghiệm $\hat{x}$ tính được bởi Gauss có thể sai lệch tới **50,000 lần** so với nghiệm thật, mặc dù residual $\|A\hat{x} - b\|_2 / \|b\|_2$ nhìn có vẻ nhỏ (cỡ 10^{-15}). Đây chính là “bẫy” kinh điển của hệ ill-conditioned: **residual nhỏ không đồng nghĩa với nghiệm đúng**. Ma trận Hilbert “chấp nhận” mọi nghiệm xấp xỉ b với residual thấp, vì bản thân nó bị kéo dãn quá nhiều trong không gian nhiều chiều.

Sự ưu việt của SVD trên ma trận Hilbert Trên cùng ma trận Hilbert $n = 500$, SVD cho residual 1.28×10^{-15} trong khi Gauss cho 2.49×10^{-14} – thấp hơn Gauss khoảng **20 lần**. Lý do nằm ở bản chất của np.linalg.lstsq: nó sử dụng **nghịch đảo giả** (pseudo-inverse) dựa trên SVD, trong đó các giá trị suy biến $\sigma_i \approx 0$ (tương ứng với chiều không gian “phẳng” của Hilbert) được **cắt tỉa** (truncated) thay vì tính nghịch đảo $1/\sigma_i \rightarrow \infty$. Cơ chế regularization ngầm này loại bỏ hoàn toàn các thành phần gây khuếch đại nhiễu, làm cho SVD trở thành phương pháp ổn định nhất trên hệ ill-conditioned [2].

Hạn chế của Gauss-Seidel trên ma trận Hilbert Gauss-Seidel báo ERR (từ chối giải) với tất cả kích thước n trên ma trận Hilbert. Nguyên nhân: ma trận Hilbert không thỏa mãn điều kiện chéo trội chặt hàng. Về lý thuyết, bán kính phổ của ma trận lặp Gauss-Seidel $\rho(G_H) \geq 1$, khiến chuỗi lặp phân kỳ. Hàm check_diagonally_dominant phát hiện điều này ngay lập tức ở bước kiểm tra đầu vào, ném ValueError trước khi thực hiện bất kỳ vòng lặp nào – đây là thiết kế bảo vệ *fail-fast* có chủ đích.

3.4 Tổng kết và đánh giá lựa chọn phương pháp

Dựa trên các kết quả thực nghiệm và phân tích lý thuyết, nhóm rút ra bảng đánh giá tổng hợp sau:

- **Khử Gauss (LU):** Là phương pháp nhanh nhất và tiết kiệm bộ nhớ nhất trong ba phương pháp trực tiếp. Hằng số $\mathcal{O}(n^3)$ thấp nhất, phù hợp làm lựa chọn mặc định cho các hệ phương trình thông thường (well-conditioned). Điểm yếu duy nhất là hoàn toàn vô lực trước ma trận điều kiện kém: residual trông nhỏ nhưng nghiệm thực tế sai xa.
- **Phân rã SVD:** Đắt đỏ nhất về thời gian thực thi (chậm hơn Gauss $\approx 7\times$ ở $n = 1000$), nhưng sở hữu khả năng “miễn nhiễm” đáng kinh ngạc với sai số số học nhờ cơ chế nghịch đảo giả và cắt tỉa giá trị suy biến. Là **lựa chọn bắt buộc** cho các hệ ill-conditioned, bài toán bình phương tối thiểu, hoặc khi ma trận có thể suy biến (rank-deficient).

Bảng 5: Bảng so sánh tổng hợp ba phương pháp giải hệ phương trình tuyến tính

Tiêu chí	Khử Gauss (LU)	Phân rã SVD	Gauss-Seidel
Độ phức tạp	$O(n^3)$	$O(n^3)$ (hằng số lớn)	$O(kn^2)$ (k nhỏ)
Tốc độ ($n = 1000$)	0.036 s	0.253 s	1.09 s
Sai số / SPD	$\sim 10^{-16}$	$\sim 10^{-15}$	$\sim 10^{-12}$
Ổn định / Hilbert	Residual nhỏ, nghiệm sai	Tốt nhất	Không áp dụng
Điều kiện	Ma trận không suy biến	Mọi ma trận	Chéo trội chặt
Ứng dụng tối ưu	Hệ thông thường	Hệ ill-conditioned	Ma trận sparse/SDD

- **Lặp Gauss-Seidel:** Tốc độ tiệm cận $\mathcal{O}(n^2)$ – vượt trội so với hai phương pháp $\mathcal{O}(n^3)$ ở kích thước lớn. Tiết kiệm bộ nhớ (không cần lưu ma trận phân rã). Tuy nhiên, điều kiện áp dụng **rất khắt khe:** chỉ hoạt động với ma trận chéo trội nghiêm ngặt, và hoàn toàn thất bại trên Hilbert. Phù hợp nhất cho bài toán vật lý kỹ thuật tự nhiên có cấu trúc sparse với tính chéo trội (ví dụ: phương trình sai phân hữu hạn cho bài toán Poisson).

4 Kết luận

4.1 Tóm tắt kết quả đạt được

Kết thúc quá trình thực hiện đề án, nhóm đã hoàn thành toàn diện 100% các mục tiêu đề ra ban đầu, bao gồm cả yêu cầu cốt lõi lẫn các phần mở rộng (điểm cộng). Các kết quả chính yếu bao gồm:

- **Cài đặt thành công Phép khử Gauss với độ ổn định cao:** Xây dựng thuật toán từ đầu (from scratch) bằng Python, áp dụng triệt để kỹ thuật Partial Pivoting. Thuật toán xử lý thành công mọi dạng ma trận (vuông, chữ nhật, suy biến) và đã được kiểm chứng tính ổn định số học tương đương với thư viện `numpy.linalg` thông qua việc triệt tiêu hiện tượng Swamping.
- **Phát triển hệ thống phân rã và chéo hóa ma trận (SVD):** Hoàn thiện thuật toán phân rã giá trị suy biến (Singular Value Decomposition) dựa trên nền tảng toán học của trị riêng và vector riêng. Thuật toán có khả năng phân rã và trích xuất đúng các ma trận U, Σ, V^T .
- **Xây dựng bộ công cụ phân tích hiệu năng (Benchmarking Suite):** Thiết lập thành công kịch bản đo lường tự động (`benchmark.py`) và hệ thống sinh dữ liệu (`data_gen.py`) để thu thập hàng ngàn điểm dữ liệu về thời gian thực thi và sai số tương đối (Residual Norm) trên đa dạng kích thước ma trận ($n \in \{50, 100, 200, 500, 1000\}$).
- **Phân tích sâu sắc tính bất ổn định số học:** Sử dụng thành công ma trận Hilbert H_n (hệ điều kiện kém) và ma trận SPD để bộc lộ hiện tượng khuếch đại sai số ở chuẩn dấu phẩy động IEEE 754. Qua đó, minh chứng rõ ràng mối liên hệ giữa Số điều kiện $\kappa_2(A)$ và mức độ "trôi dạt" của vector nghiệm.
- **Trực quan hóa Toán học sinh động bằng Manim:** Chuyển hóa các phương trình đại số tuyến tính khô khan thành Video hoạt hình (Animation) chất lượng cao. Video mô phỏng trực quan các phép biến đổi hình học của ma trận trong không gian, giúp giải thích trực diện bản chất của quá trình phân rã.
- **Quy trình phát triển phần mềm chuyên nghiệp:** Ứng dụng triệt để Git/GitHub trong quản lý mã nguồn nhóm, cấu trúc thư mục module hóa (`solvers`, `config`), kết hợp Jupyter Notebook và $\text{\LaTeX}$ để xuất bản báo cáo học thuật đạt chuẩn.
- Cài đặt thêm phương pháp lặp **Gauss-Seidel** với cơ chế tự động kiểm tra điều kiện hội tụ (ma trận chéo trội chặt hàng).
- Xây dựng kịch bản benchmark để đo lường thời gian thực thi trên các kích thước ma trận lớn ($n \in \{50, 100, 200, 500, 1000\}$). Vẽ đồ thị *log-log* so sánh và chứng minh thực nghiệm chi phí tính toán của các phương pháp trực tiếp hoàn toàn bám sát đường lý thuyết $O(n^3)$.
- Nghiên cứu thành công sự bất ổn định số học: Sử dụng **Số điều kiện $\kappa_2(A)$** kết hợp đối chiếu

giữa ma trận Hilbert H_n (ill-conditioned) và ma trận SPD ngẫu nhiên để bộc lộ hiện tượng khuếch đại sai số, giải thích sâu sắc giới hạn của hệ thống dấu phẩy động IEEE 754.

4.2 Bài học rút ra

Quá trình thực hiện đồ án không chỉ là việc hiện thực hóa các công thức toán học lên ngôn ngữ lập trình, mà còn là một hành trình giúp nhóm thay đổi tư duy về khoa học tính toán:

- **Sự khác biệt giữa Toán học lý thuyết và Tính toán số học:** Nhóm nhận ra rằng một thuật toán đúng trên giấy chưa chắc đã chạy đúng trên máy tính. Việc hiểu về cấu trúc lưu trữ số thực *double precision* là chìa khóa để giải thích tại sao $0.1 + 0.2$ không bao giờ bằng 0.3 tuyệt đối trong RAM.
- **Tư duy phản biện qua dữ liệu:** Thay vì chấp nhận sai số, nhóm đã học được cách đặt câu hỏi “Tại sao?”. Việc quan sát đường sai số bám sát ngưỡng 10^{-16} đã giúp nhóm tin tưởng vào thuật toán của mình hơn là việc chỉ nhìn vào kết quả nghiệm cuối cùng.
- **Kỹ năng tối ưu hóa quy trình làm việc:** Nhóm đã làm quen với việc tự động hóa (Automation) thông qua *latexmk* và các script *benchmark*. Bài học về việc “lười biếng một cách thông minh” – viết script để máy tính tự đo đạc hàng ngàn phép tính – đã giúp nhóm tiết kiệm rất nhiều thời gian.
- **Giá trị của việc chuẩn hóa (Standardization):** Việc thống nhất sử dụng `config.py` và quy chuẩn `list[list[float]]` ngay từ đầu là bài học xương máu về quản lý dự án, giúp việc ghép code giữa các thành viên diễn ra trơn tru mà không bị xung đột kiểu dữ liệu.

4.3 Khó khăn chuyên môn và các nghịch lý Toán học tính toán

Trong suốt quá trình thực hiện đồ án, rào cản lớn nhất của nhóm không nằm ở cú pháp lập trình, mà nằm ở sự sai khác giữa Toán học lý thuyết (trên giấy) và Giải tích số (trên máy tính). Dưới đây là bốn khó khăn cốt lõi và bài học nhận thức nhóm đã rút ra.

1. Giới hạn của chuẩn IEEE 754

Hiện tượng: Trong giai đoạn kiểm thử ban đầu, nhóm kỳ vọng sai số phần dư sẽ bằng 0.0 tuyệt đối. Tuy nhiên, khi giải các hệ phương trình chứa phân số (như $\frac{1}{3}$) hay số thập phân (như 0.1), sai số phần dư luôn trôi nổi ở mức 10^{-16} .

Bản chất Toán học: Đây không phải là lỗi thuật toán, mà là giới hạn biểu diễn tất yếu của chuẩn dấu phẩy động IEEE 754 (Double Precision) [6]. Các phân số vô hạn tuần hoàn trong hệ cơ số 2 bị cắt cụt ở bit thứ 53, sinh ra nhiễu làm tròn (round-off error). Ranh giới này được gọi là *Machine Epsilon*: $\epsilon_{\text{machine}} \approx 2.22 \times 10^{-16}$.

Giải quyết: Nhóm phải từ bỏ tư duy ”so sánh bằng 0” (exact equality) của Toán học. Thay vào đó, hằng số $\text{EPSILON} = 1\text{e-}12$ được thiết lập tập trung trong `config.py` làm ngưỡng dung sai (tolerance), cho phép các thuật toán phân rã SVD và Jacobi hội tụ một cách an toàn mà không bị ảnh hưởng bởi nhiễu máy tính.

2. Sai số phần dư trên ma trận Hilbert

Hiện tượng: Khi đưa ma trận Hilbert H_n (với $n \geq 50$) vào thuật toán Gauss, máy tính trả về vector nghiệm $\hat{x}$ sai lệch hoàn toàn so với nghiệm lý thuyết. Điều kỳ lạ là khi kiểm tra lại bằng sai số phần dư $\|A\hat{x} - b\|_2$, kết quả vẫn cực kỳ nhỏ (cỡ 10^{-15}) – máy tính báo “đúng” nhưng nghiệm thực tế lại “sai hoàn toàn”.

Bản chất Toán học: Nhóm đã vấp phải nghịch lý kinh điển của hệ điều kiện kém (ill-conditioned system). Ma trận Hilbert có số điều kiện $\kappa_2(A)$ khổng lồ, lên tới 10^{20} . Theo Định lý sai số [3]:

$$\frac{\|\Delta x\|}{\|x\|} \leq \kappa_2(A) \cdot \frac{\|\Delta b\|}{\|b\|}$$

Chỉ cần một hạt nhiễu $\epsilon_{\text{machine}}$ rất yếu trong vế phải b , sai số thực tế (forward error) của nghiệm sẽ bị khuếch đại lên $\kappa_2(A)$ lần. Sai số phần dư nhỏ chỉ là một ảo ảnh toán học do không gian bị kéo dãn quá mức – đây là “bẫy” đặc trưng của hệ ill-conditioned mà bất kỳ nhà giải tích số nào cũng phải cảnh giác.

Giải quyết: Sự kiện này buộc nhóm phải nhìn nhận lại giá trị thực sự của thuật toán SVD. Nhờ khả năng dùng nghịch đảo giả (pseudo-inverse) và cắt tỉa các giá trị suy biến tiệm cận 0, SVD đã chứng minh được đây là công cụ duy nhất “miễn nhiễm” một phần với sự bùng nổ sai số trên hệ điều kiện kém.

3. Bức tường của hằng số $\mathcal{O}(n^3)$ và nút thắt hiệu năng Python

Hiện tượng: Trong lý thuyết, cả Khử Gauss và SVD đều có độ phức tạp $\mathcal{O}(n^3)$. Tuy nhiên, khi nhóm chạy benchmark bằng Python thuần ở $n = 500, 1000$, chương trình bị treo (Timeout > 60 s). Trong khi đó, `np.linalg.solve` của NumPy giải quyết $n = 1000$ chỉ trong 0.036 giây.

Bản chất Toán học và Kiến trúc: Độ phức tạp $\mathcal{O}(n^3)$ chỉ mô tả tốc độ tăng trưởng – nó che giấu một hằng số thực thi C rất lớn. Thuật toán SVD (Jacobi/QR) đòi hỏi nhiều vòng quét, phép tính lượng giác và căn bậc hai hơn hẳn Gauss, khiến $C_{\text{SVD}} \gg C_{\text{Gauss}}$. Bên cạnh đó, rào cản của trình thông dịch Python (interpreted) khiến phần cứng không thể song song hóa các phép nhân ma trận để phát huy tối đa khả năng của CPU.

Giải quyết: Dưới sự cho phép của giảng viên, nhóm đã linh hoạt chuyển sang sử dụng backend BLAS/LAPACK thông qua NumPy cho các kích thước n lớn. Quyết định này đảm bảo thu thập đủ số liệu để vẽ đồ thị Log-Log chuẩn xác và minh chứng được sự song song giữa đường thực nghiệm với đường tiệm cận $\mathcal{O}(n^3)$ lý thuyết.

4. Ranh giới hội tụ khắt khe của phương pháp lặp Gauss-Seidel

Hiện tượng: Ban đầu, khi thử nghiệm Gauss-Seidel trên các ma trận SPD ngẫu nhiên, thuật toán liên tục báo ERR (vượt quá số vòng lặp tối đa mà không hội tụ), mặc dù lý thuyết khẳng định Gauss-Seidel chắc chắn hội tụ trên ma trận SPD.

Bản chất Toán học: Định lý chỉ khẳng định ma trận SPD sẽ hội tụ, nhưng không hứa hẹn tốc độ hội tụ. Nếu bán kính phổ (spectral radius) của ma trận lặp $\rho(G)$ nằm quá sát 1, phương pháp lặp tiến triển cực kỳ chậm và cạn kiệt giới hạn vòng lặp cho phép trước khi đạt được ngưỡng hội tụ.

Giải quyết: Nhóm đã can thiệp vào khâu sinh dữ liệu (`data_gen.py`), bắt buộc mọi ma trận SPD phải được cộng thêm biên độ vào đường chéo chính để thỏa mãn điều kiện **chéo trội nghiêm ngặt** (Strictly Diagonally Dominant):

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i = 1, \dots, n$$

Chỉ khi ép chặt điều kiện này, bán kính phổ $\rho(G)$ mới thực sự nhỏ hơn 1 đủ nhiều, giúp Gauss-Seidel hội tụ ổn định và phát huy sức mạnh tốc độ $\mathcal{O}(n^2)$ đặc trưng của phương pháp lặp.

Tài liệu

- [1] Gilbert Strang, *Introduction to Linear Algebra*, 5th Edition, Wellesley-Cambridge Press, 2016.
- [2] Gene H. Golub và Charles F. Van Loan, *Matrix Computations*, 4th Edition, Johns Hopkins University Press, 2013.
- [3] Lloyd N. Trefethen và David Bau III, *Numerical Linear Algebra*, SIAM, 1997.
- [4] Gilbert Strang, *MIT 18.06SC Linear Algebra*, MIT OpenCourseWare, <https://ocw.mit.edu/courses/18-06sc-linear-algebra-fall-2011/>, truy cập ngày 14/04/2026.
- [5] Virginia C. Klema và Alan J. Laub, *The singular value decomposition: Its computation and some applications*, IEEE Transactions on Automatic Control, Vol. 25, No. 2, pp. 164–176, 1980.
- [6] IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*, <https://ieeexplore.ieee.org/document/8766229>, truy cập ngày 09/04/2026.
- [7] Nicholas J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd Edition, SIAM, 2002.
- [8] David Goldberg, *What Every Computer Scientist Should Know About Floating-Point Arithmetic*, ACM Computing Surveys, Vol. 23, No. 1, pp. 5–48, 1991.
- [9] NumPy Developers, *NumPy v1.26 Manual – Linear Algebra (numpy.linalg)*, <https://numpy.org/doc/stable/reference/routines.linalg.html>, truy cập ngày 09/04/2026.
- [10] GeeksforGeeks, *Gaussian Elimination*, <https://www.geeksforgeeks.org/gaussian-elimination/>, truy cập ngày 14/04/2026.
- [11] Manim Community Developers, *Manim Community Documentation v0.18.0*, <https://docs.manim.community/>, truy cập ngày 09/04/2026.

A Phụ lục

A.1 Bảng số liệu chi tiết

Dưới đây là bảng số liệu chi tiết thu thập được trong quá trình thực nghiệm:

STT	Tham số	Giá trị đo được
1	Tham số A	10.5

Bảng 6: Bảng số liệu chi tiết

A.2 Code bổ sung

Đoạn mã nguồn quan trọng được sử dụng trong hệ thống:

```
1 import numpy as np
2
3 def process_data(data_path):
4     print("Code minh hoa chen tai phu luc...")
5     return True
```

Đoạn mã 16: Mã nguồn xử lý dữ liệu (Python)